

**LAPORAN PROJECT AKHIR ARTIFICIAL INTELLIGENCE
PREDIKSI HARGA RUMAH DENGAN ALGORITMA RANDOM FOREST
XG BOOST, LIGHT GRADIENT BOOSTING MACHINE**



Dosen Pengampu Mata Kuliah:

Riskyana Dewi Intan Puspitasari, M.Kom.
Yuni Rosita Dewi, M.Si.

Disusun Oleh:

Fitri Saidatur Hanifah	24031554035
Meida Rifa Alfiani	24031554139
Darista Wardhani	24031554146

**PRODI S1 SAINS DATA
FAKULTAS MATEMATIKA & ILMU PENGETAHUAN ALAM
UNIVERSITAS NEGERI SURABAYA
SEMESTER GASAL 2025/2026**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Harga properti seperti rumah selalu memiliki perubahan harga setiap harinya, perubahan harga ini ditentukan dari berbagai aspek seperti lokasi rumah, luas bangunan, akses menuju fasilitas umum. Selain itu, faktor eksternal seperti pertumbuhan populasi, tingkat urbanisasi, dan kondisi pasar properti juga berperan penting dalam menentukan fluktuasi harga tersebut. Kompleksitas faktor-faktor ini menyebabkan penentuan harga rumah tidak selalu mudah dan seringkali membutuhkan analisis mendalam agar nilainya dapat mencerminkan kondisi yang sebenarnya.

Dalam praktiknya, penjual, pembeli, maupun agen properti membutuhkan sebuah sistem penilaian harga yang akurat dan standarnya dapat dipertanggungjawabkan. Penilaian yang tidak tepat dapat mengakibatkan kerugian, baik karena harga yang ditawarkan terlalu rendah maupun terlalu tinggi dibandingkan nilai pasar. Oleh karena itu, untuk menentukan harga rumah yang memiliki sistem standar pembandingan sama diperlukan data historis untuk menguji sistem prediksi memiliki keluaran yang relevan dengan kondisi rumah tersebut.

Sistem prediksi dapat memanfaatkan teknik machine learning, menawarkan solusi yang efektif untuk membangun sistem prediksi harga rumah yang lebih akurat dan adaptif. Melalui algoritma seperti regresi linear, random forest, atau gradient boosting, sistem dapat mempelajari hubungan kompleks antara fitur-fitur rumah dan harga pasar aktual. Model prediksi ini tidak hanya membantu dalam estimasi harga, tetapi juga memungkinkan pengguna untuk memahami faktor apa saja yang paling berpengaruh terhadap nilai properti.

Dengan demikian, pembangunan sistem prediksi harga rumah berbasis machine learning memiliki peran penting dalam meningkatkan transparansi, efisiensi, dan akurasi dalam proses penilaian properti. Sistem ini dapat menjadi alat bantu strategis bagi berbagai pihak —mulai dari calon pembeli, investor, hingga pengembang perumahan— untuk membuat keputusan yang lebih tepat dan berbasis data.

1.2 Tujuan

- 1.2.1 Menerapkan algoritma Random Forest untuk membangun model prediksi harga rumah.
- 1.2.2 Menyediakan sistem prediksi harga rumah yang memperkirakan harga berdasarkan fitur.
- 1.2.3 Membandingkan performa beberapa algoritma machine learning untuk menentukan metode yang paling efektif digunakan dalam prediksi harga rumah.
- 1.2.4 Menghasilkan model yang dapat digunakan sebagai standar pembandingan harga sehingga proses penilaian properti menjadi lebih transparan dan berbasis data.

1.3 Manfaat

- 1.3.1 Bagi penjual dan agen properti dapat membentuk strategi harga yang kompetitif dan realistis berdasarkan data pasar

- 1.3.2 Mempermudah pemerintah atau lembaga bisnis untuk mendukung analisis nilai lahan, zonasi, dan pembuatan kebijakan perumahan.
- 1.3.3 Menyediakan informasi prediktif yang berguna untuk menganalisis peluang investasi properti.
- 1.3.4 Pembeli diharapkan mampu memperkirakan harga rumah yang sesuai dengan kondisi pasar sehingga dapat membuat keputusan yang lebih tepat.

BAB II PENDAHULUAN

2.1 Dataset

Dataset yang kami gunakan berupa dataset public yang bersumber dari laman Kagle dengan judul dataset House Sale Data. Dataset ini mencakup file train.csv yang memiliki 1168 baris dan 81 kolom (fitur). Dataset ini yang nantinya akan dilakukan eksplorasi data awal untuk melihat apakah dataset ini memenuhi informasi yang kami butuhkan untuk melakukan Prediksi harga rumah. Berikut 81 fitur yang ada pada dataset House Sale Data:

- a. Identitas
 - ID: Nomor unik tiap observasi/rumah
- b. Informasi rumah & lahan
 - MSSubClass = Kategori jenis bangunan perumahan.
 - MSZoning = Zona penggunaan lahan (residential, commercial).
 - LotFrontage = Panjang lahan bagian depan rumah (dalam feet).
 - LotArea = Luas total tanah.
 - Street = Jenis jalan menuju rumah (paved/gravel).
 - Alley = Jenis gang
 - LotShape = Bentuk lahan (regular/irregular).
 - LandContour = Kontur tanah (datar, rendah, dsb).
 - Utilities = Ketersediaan utilitas dasar.
 - LotConfig = Konfigurasi lot/posisi rumah di lahan.
 - LandSlope = Kemiringan lahan.
 - Neighborhood = Nama kawasan/lingkungan.
 - Condition1 = Kondisi sekitar (misalnya dekat jalan utama).
 - Condition2 = Kondisi tambahan sekitar.
- c. Tipe bangunan & gaya rumah
 - BldgType = Tipe bangunan (single-family, duplex, dsb).
 - HouseStyle = Gaya rumah (1 lantai, 2 lantai, split level).
- d. Kualitas & kondisi
 - OverallQual = Kualitas keseluruhan material rumah (1–10).
 - OverallCond = Kondisi keseluruhan rumah (1–10).
- e. Tahun dibangun & Renovasi
 - YearBuilt = Tahun rumah dibangun.
 - YearRemodAdd = Tahun renovasi besar dilakukan.
- f. Atap & Eksterior
 - RoofStyle = Gaya atap (gable, hip, dll).
 - RoofMatl = Material atap.

- Exterior1st = Material luar bangunan (lapisan pertama).
- Exterior2nd = Material luar bangunan lapisan kedua.
- ExterQual = Kualitas eksterior.
- ExterCond = Kondisi eksterior.

g. Sistem pemanas & AC

- Foundation = Jenis pondasi rumah.
- Heating = Jenis sistem pemanas.
- HeatingQC = Kualitas pemanas.
- CentralAir = Ada AC (Y/N).
- Electrical = Jenis sistem listrik.
- Basement (Ruang Bawah Tanah)
- BsmtQual = Kualitas basement.
- Kondisi basement.
- BsmtExposure = Basement menghadap keluar atau tidak.
- BsmtFinType1 = Tipe finishing basement 1.
- BsmtFinSF1 = Luas finishing basement 1.
- BsmtFinType2 = Tipe finishing basement 2.
- BsmtFinSF2 = Luas finishing basement 2.
- BsmtUnfSF = Luas basement yang belum di-finishing.
- TotalBsmtSF = Total luas basement.

h. Struktur Lantai dan Luas Bangunan

- 1stFlrSF = Luas lantai pertama.
- 2ndFlrSF = Luas lantai kedua.
- LowQualFinSF = Luas finishing kualitas rendah.
- GrLivArea = Total luas ruang tinggal di atas permukaan tanah.

i. Kamar Mandi dan Jumlah Ruangan

- BsmtFullBath = Jumlah kamar mandi penuh di basement.
- BsmtHalfBath = Jumlah kamar mandi setengah di basement.
- FullBath = Jumlah kamar mandi penuh di lantai atas.
- HalfBath = Jumlah kamar mandi setengah di lantai atas.
- BedroomAbvGr = Jumlah kamar tidur di atas tanah.
- KitchenAbvGr = Jumlah dapur.
- KitchenQual = Kualitas dapur.
- TotRmsAbvGrd = Total jumlah ruangan di atas permukaan tanah.
- Functional = Tingkat fungsi rumah (normal/tidak normal).

j. Fitur Tambahan Interior

- Fireplaces = Jumlah perapian.
- FireplaceQu = Kualitas perapian.

k. Garasi

- GarageType = Tipe garasi.
- GarageYrBlt = Tahun garasi dibangun.
- GarageFinish = Tipe finishing garasi.
- GarageCars = Kapasitas garasi (mobil).
- GarageArea = Luas garasi.
- GarageQual = Kualitas garasi.
- GarageCond = Kondisi garasi.

l. Area Luar dan Fasilitas Lainnya

- PavedDrive = Jalan masuk yang beraspal (Y/N/P).
- WoodDeckSF = Luas dek kayu.
- OpenPorchSF = Luas teras terbuka.
- EnclosedPorch = Luas teras tertutup.
- 3SsnPorch = Luas teras tiga musim.
- ScreenPorch = Luas teras dengan layar.
- PoolArea = Luas kolam renang.
- PoolQC = Kualitas kolam renang.
- Fence = Jenis pagar.
- MiscFeature = Fitur tambahan lainnya.
- MiscVal = Nilai fitur tambahan.

m. Informasi Penjualan

- MoSold = Bulan penjualan.
- YrSold = Tahun penjualan.
- SaleType = Tipe penjualan.
- SaleCondition = Kondisi penjualan.
- SalePrice = Harga jual rumah.

2.2 Eksplorasi Data Analisis (EDA) awal

a. Membaca dataset *train.csv*

Program berhasil membaca file dan mengintegrasikan seluruh data ke dalam sebuah DataFrame bernama *df*. Hasil integrasi ditunjukkan melalui keluarnya informasi jumlah baris dan kolom, yaitu 1460 baris dan 81 kolom, yang menandakan bahwa seluruh data pada file CSV telah berhasil dimuat tanpa error. Selanjutnya, tampilan lima baris pertama menunjukkan contoh isi dataset seperti kolom *Id*, *MSSubClass*, *MSZoning*, *LotFrontage*, hingga *SalePrice*. Hal ini menegaskan bahwa proses pemanggilan dan pembacaan data telah berjalan dengan baik, karena setiap kolom muncul sesuai struktur asli file.

Jumlah baris dan kolom: (1460, 81)

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType	SaleCondition	SalePrice
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2008	WD	Normal	208500
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5	2007	WD	Normal	181500
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	9	2008	WD	Normal	223500
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2006	WD	Abnorml	140000
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	12	2008	WD	Normal	250000

5 rows x 81 columns

b. `df.info()`

Sistem berhasil menampilkan jenis data pada setiap kolom, jumlah nilai non-null, serta tipe data masing-masing variabel. Informasi ini memperlihatkan bahwa struktur dataset telah terintegrasi sepenuhnya, termasuk apakah suatu kolom berupa angka, teks, atau kategori.

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1460 entries, 0 to 1459

Data columns (total 81 columns):

#	Column	Non-Null Count	Dtype
0	Id	1460 non-null	int64
1	MSSubClass	1460 non-null	int64
2	MSZoning	1460 non-null	object
3	LotFrontage	1201 non-null	float64
4	LotArea	1460 non-null	int64
5	Street	1460 non-null	object
6	Alley	91 non-null	object
7	LotShape	1460 non-null	object
8	LandContour	1460 non-null	object
9	Utilities	1460 non-null	object
10	LotConfig	1460 non-null	object
11	Landslope	1460 non-null	object
12	Neighborhood	1460 non-null	object
13	Condition1	1460 non-null	object
14	Condition2	1460 non-null	object
15	BldgType	1460 non-null	object
16	HouseStyle	1460 non-null	object
17	OverallQual	1460 non-null	int64
18	OverallCond	1460 non-null	int64
19	YearBuilt	1460 non-null	int64
20	YearRemodAdd	1460 non-null	int64
21	RoofStyle	1460 non-null	object
22	RoofMatl	1460 non-null	object
23	Exterior1st	1460 non-null	object
24	Exterior2nd	1460 non-null	object
25	MasVnrType	588 non-null	object
26	MasVnrArea	1452 non-null	float64
27	ExterQual	1460 non-null	object
28	ExterCond	1460 non-null	object
29	Foundation	1460 non-null	object
30	BsmtQual	1423 non-null	object
31	BsmtCond	1423 non-null	object
32	BsmtExposure	1422 non-null	object
33	BsmtFinType1	1423 non-null	object
34	BsmtFinSF1	1460 non-null	int64
35	BsmtFinType2	1422 non-null	object
36	BsmtFinSF2	1460 non-null	int64
37	BsmtUnfSF	1460 non-null	int64
38	TotalBsmtSF	1460 non-null	int64
39	Heating	1460 non-null	object
40	HeatingQC	1460 non-null	object
41	CentralAir	1460 non-null	object
42	Electrical	1459 non-null	object
43	1stFlrSF	1460 non-null	int64
44	2ndFlrSF	1460 non-null	int64
45	LowQualFinSF	1460 non-null	int64
46	GrLivArea	1460 non-null	int64
47	BsmtFullBath	1460 non-null	int64
48	BsmtHalfBath	1460 non-null	int64
49	FullBath	1460 non-null	int64
50	HalfBath	1460 non-null	int64
51	BedroomAbvGr	1460 non-null	int64
52	KitchenAbvGr	1460 non-null	int64

c. `df.describe()`

Fungsi ini bertujuan untuk mendapatkan ringkasan statistik dari variabel numerik. Hasil yang ditampilkan menunjukkan nilai-nilai seperti *mean*, *median*, *minimum*, *maximum*, hingga *standard deviation* untuk setiap kolom numerik. Informasi ini menandakan bahwa dataset telah berhasil terintegrasi dan dapat dianalisis secara statistik. Misalnya, kolom *LotArea*, *OverallQual*, dan *YearBuilt* memiliki ringkasan yang lengkap, sehingga seluruh nilai numerik dapat diolah dengan benar.

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BasFinSF1	...	WoodDeckSF	OpenPorchSF	EnclosedPorch	3SeasonPorch	ScreenPorch	PoolArea	MiscVal	MoSold	YrSold	SalePrice
count	1460.000000	1460.000000	1201.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1462.000000	1460.000000	...	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000
mean	730.500000	56.897260	70.049950	10516.820002	6.099315	5.575342	1971.267000	1984.865753	103.685262	443.639726	...	94.244521	46.660274	21.954110	3.409589	15.060959	2.758904	43.489041	6.321918	2007.815753	180921.195890
std	421.610009	42.300571	24.284752	9861.264932	1.382997	1.112799	30.202904	20.645407	181.066207	456.090091	...	125.338794	66.256028	61.119149	29.317331	55.757415	40.177307	496.123024	2.703626	1.328095	79442.502883
min	1.000000	20.000000	21.000000	1300.000000	1.000000	1.000000	1872.000000	1950.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	2006.000000	34900.000000	
25%	365.750000	20.000000	59.000000	7553.500000	5.000000	5.000000	1954.000000	1967.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	5.000000	2007.000000	129875.000000	
50%	730.500000	50.000000	69.000000	9478.500000	6.000000	5.000000	1973.000000	1994.000000	0.000000	383.500000	...	0.000000	25.000000	0.000000	0.000000	0.000000	0.000000	6.000000	2008.000000	163000.000000	
75%	1095.250000	70.000000	80.000000	11601.500000	7.000000	6.000000	2000.000000	2004.000000	166.000000	712.250000	...	168.000000	68.000000	0.000000	0.000000	0.000000	0.000000	8.000000	2009.000000	214000.000000	
max	1460.000000	190.000000	313.000000	215245.000000	10.000000	9.000000	2010.000000	2010.000000	1600.000000	5644.000000	...	857.000000	547.000000	552.000000	508.000000	480.000000	738.000000	15500.000000	12.000000	2010.000000	755000.000000

8 rows × 30 columns

d. `df.isnull().sum()`

Fungsi ini berguna untuk menghitung persentase missing value dengan memanfaatkan fungsi `df.isnull().mean() * 100`. Hasilnya kemudian disusun dalam sebuah DataFrame baru bernama *missing_df*. Data ini menunjukkan seberapa banyak nilai kosong pada setiap kolom, sekaligus urutannya berdasarkan persentase paling tinggi. Dari perintah tersebut akan menghasilkan output beberapa kolom memiliki tingkat missing value yang sangat besar, misalnya:

- PoolQC dengan 99.52% data hilang
- MiscFeature dengan 96.30% data hilang
- Alley dengan 93.77% data hilang
- Fence dengan 81.54% data hilang

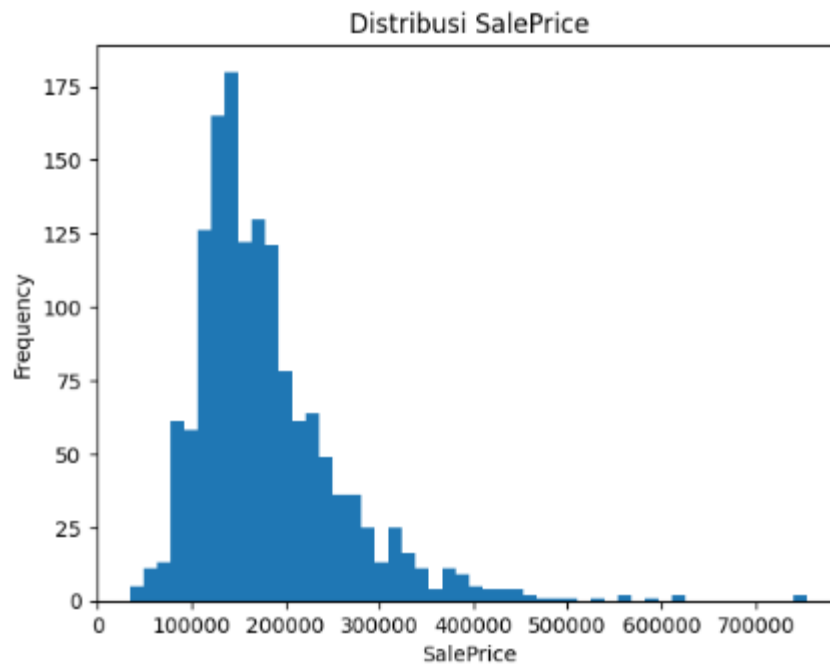
Sebaliknya, ada pula kolom-kolom yang tidak memiliki missing value sama sekali seperti *MoSold*, *YrSold*, *SaleType*, dan *SaleCondition*. Hal ini membuktikan bahwa integrasi data sudah berhasil dilakukan, karena program mampu menghitung dan menampilkan jumlah serta persentase missing value untuk setiap variabel.

	Missing Count	Missing %
PoolQC	1453	99.52
MiscFeature	1406	96.30
Alley	1369	93.77
Fence	1179	80.75
MasVnrType	872	59.73
...	...	...
MoSold	0	0.00
YrSold	0	0.00
SaleType	0	0.00
SaleCondition	0	0.00
SalePrice	0	0.00

81 rows × 2 columns

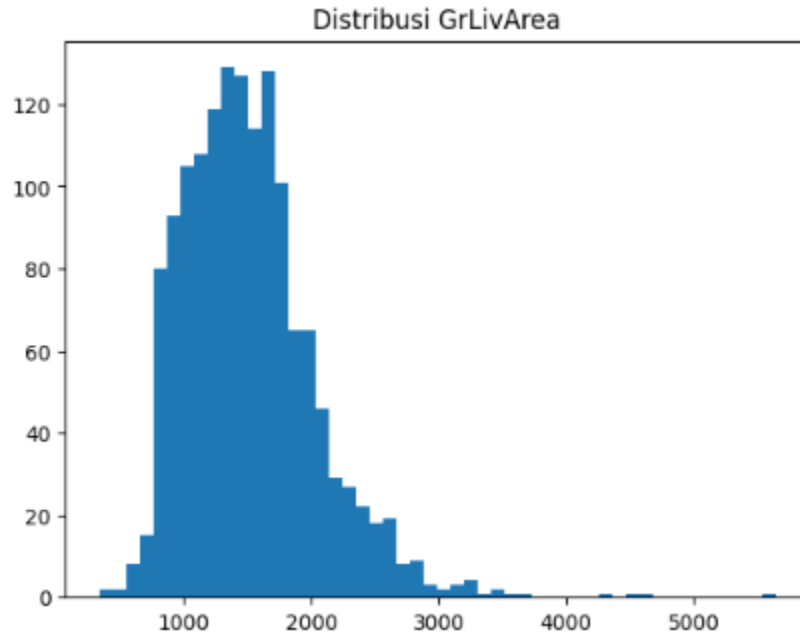
e. Grafik histogram

Script code yang dijalankan akan menghasilkan grafik histogram yang memperlihatkan bahwa nilai SalePrice memiliki distribusi yang condong ke kanan (skewed right). Sebagian besar rumah memiliki harga pada rentang sekitar 100.000 hingga 250.000, dan semakin sedikit rumah yang memiliki harga lebih tinggi. Hal ini memberikan gambaran awal bahwa harga rumah di dataset tidak tersebar secara merata, dan mayoritas berada pada level harga menengah.



f. Distribusi variabel GrLivArea

Script code akan menampilkan distribusi variable luas area hunian yang digunakan. Distribusinya juga terlihat agak condong ke kanan, dengan sebagian besar rumah memiliki luas antara 1000 hingga 2000 square feet. Hanya sedikit rumah yang memiliki area lebih besar dari itu. Ini menandakan bahwa sebagian besar rumah dalam dataset memiliki ukuran standar, dengan beberapa outlier di luas yang besar.



g. `Df.corr(numeric_only = True ["SalePrice"]).sort_values(ascending = False)`

Fungsi `df.corr` dengan variable pendukungnya digunakan untuk menghitung matriks korelasi untuk semua kolom numerik dalam dataset `train.csv`. Korelasi menunjukkan seberapa kuat hubungan linear antara dua variabel. Fitur yang paling menentukan harga rumah yaitu `OverallQual`, `GrLivArea`, `GarageCars` & `GarageArea`, `TotalBsmtSF`. Harga rumah sangat kuat dipengaruhi oleh faktor tersebut karena memiliki korelasi tinggi yaitu $>0,6$. Selanjutnya, fitur berikut memberi kontribusi tetapi tidak sekuat kelompok pertama yaitu termasuk korelasi menengah dengan range $(0,4-0,59)$ diantaranya adalah `FullBath`, `TotRmsAbvGrd`, `YearBuilt`, `YearRemodAdd`, `GarageYrBlt`, `MasVnrArea`, `Fireplaces`. Selanjutnya dilanjutkan dengan fitur korelasi rendah tapi positif dengan range $(0,1-0,39)$ yang artinya berpengaruh tapi tidak besar. Kategori fitur berikutnya yaitu korelasi sangat rendah dengan range $(0,0-0,15)$. Selanjutnya kategori terakhir yaitu korelasi negatif yaitu fitur dengan range $<0,15$ dimana fitur yang cenderung menurunkan harga.

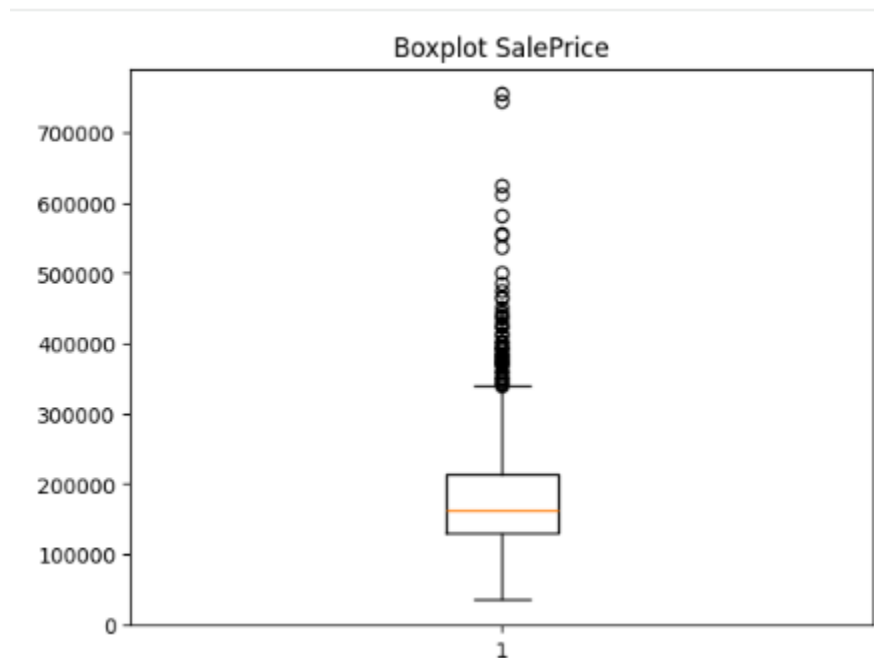
	SalePrice
• SalePrice	1.000000
OverallQual	0.790982
GrLivArea	0.708624
GarageCars	0.640409
GarageArea	0.623431
TotalBsmtSF	0.613581
1stFlrSF	0.605852
FullBath	0.560664
TotRmsAbvGrd	0.533723
YearBuilt	0.522897
YearRemodAdd	0.507101
GarageYrBlt	0.486362
MasVnrArea	0.477493
Fireplaces	0.466929
BsmtFinSF1	0.386420
LotFrontage	0.351799
WoodDeckSF	0.324413
2ndFlrSF	0.319334
OpenPorchSF	0.315856
HalfBath	0.284108
LotArea	0.263843
BsmtFullBath	0.227122
BsmtUnfSF	0.214479
BedroomAbvGr	0.168213
ScreenPorch	0.111447
PoolArea	0.092404
MoSold	0.046432
3SsnPorch	0.044584
BsmtFinSF2	-0.011378
BsmtHalfBath	-0.016844
MiscVal	-0.021190

h. Grafik boxplot

Script code akan menampilkan persebaran harga rumah (*SalePrice*) beserta nilai ekstrem dan outlier. Dari gambar terlihat bahwa:

- Rentang harga rumah berada pada kisaran yang cukup luas.
- Terdapat banyak outlier di area harga tinggi, ditunjukkan oleh titik-titik di atas whisker boxplot.
- Sebagian besar harga rumah terkonsentrasi pada bagian bawah rentang, menunjukkan persebaran yang tidak merata.

Visualisasi ini menegaskan bahwa dataset telah berhasil diproses dan cocok untuk analisis deteksi outlier atau transformasi data.



2.3 Preprocessing (Data Cleaning)

a. Seluruh Fitur

Dilakukan pembangunan model dengan data training tanpa melakukan filter kolom, secara sederhana model di training dengan seluruh kolom yang ada pada dataset training. Dalam preprocessing script akan melakukan transformer menggunakan scaler pada kolom numeric dan One Hot Encoding pada kolom kategorikal. Berikut script yang dijalankan dan file CSV yang dihasilkan dari langkah preprocessing:

```

from sklearn.pipeline import Pipeline
numerical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

```

train_preprocessed.csv

1 to 10 of 1460 entries

Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearB
-1.730864877400689	0.07337496353744775	-0.22087508895451458	-0.20714170777431132	0.6514792433257054	-0.5171998069472914	1.050993788
-1.728492204505006	-0.8725627562389217	0.4603197354510445	-0.09188637231949037	-0.07183611428306244	2.179627757849301	0.156733711
-1.7261195316093232	0.07337496353744775	-0.08463612407340275	0.07347997855047012	0.6514792433257054	-0.5171998069472914	0.984752301
-1.7237468587136402	0.3098593934815401	-0.44794003042303426	-0.09689747386100432	0.6514792433257054	-0.5171998069472914	-1.86363165
-1.7213741858179572	0.07337496353744775	0.6419716886258603	0.37514829134961014	1.3747946009344731	-0.5171998069472914	0.951631551
-1.7190015129222744	-0.1631094664066446	0.6873846769195642	0.3606160968792197	-0.7951514718918303	-0.5171998069472914	0.719786351
-1.7166288400265914	-0.8725627562389217	0.23325479398252483	-0.04337890939763529	1.3747946009344731	-0.5171998069472914	1.084114532
-1.7142561671309084	0.07337496353744775	-0.03922313577969881	-0.013512744210212127	0.6514792433257054	0.38174271465157283	0.057371475
-1.7118834942352257	-0.1631094664066446	-0.8566569250663697	-0.44065903960886155	0.6514792433257054	-0.5171998069472914	-1.33369975
-1.7095108213395427	3.147672552810648	-0.9020699133600736	-0.31037039952949874	-0.7951514718918303	0.38174271465157283	-1.06873380

Show 10 per page

1 2 10 100 140 146

b. 12 Fitur Berdasarkan Lokasi

Preprocessing melakukan proses data wrangling berupa pemilihan subset kolom, pembersihan missing value untuk kolom numerik dengan median. Median dipilih karena lebih tahan terhadap outlier dibanding mean dan pengisian missing value pada kolom kategorikal dengan label khusus yaitu menjadi “Unknown” Hal ini berguna untuk mencegah error saat encoding (misalnya ketika one-hot encoding). Setelah data bersih dan konsisten, dataset disimpan sebagai cleaned_data.csv untuk keperluan analisis lebih lanjut.

```
import pandas as pd

df = pd.read_csv('train.csv')
selected_cols = [
    'Neighborhood', 'MSZoning', 'Street', 'LotConfig',
    'Condition1', 'Condition2', 'Alley', 'OverallQual',
    'LandContour', 'LandSlope', 'LotFrontage', 'LotArea', 'SalePrice'
]

data = df[selected_cols].copy()
data['LotFrontage'] = data['LotFrontage'].fillna(data['LotFrontage'].median())
data['Alley'] = data['Alley'].fillna('None')

categorical_cols = ['Neighborhood', 'MSZoning', 'Street', 'LotConfig',
                    'Condition1', 'Condition2', 'LandContour', 'LandSlope']

for col in categorical_cols:
    data[col] = data[col].fillna('Unknown')

print(data.head())
data.to_csv('cleaned_data.csv', index=False)
```

Neighborhood	MSZoning	Street	LotConfig	Condition1	Condition2	Alley	OverallQual	LandContour	LandSlope	LotFrontage	LotArea	SalePrice
CollgCr	RL	Pave	Inside	Norm	Norm	None	7	Lvl	Gtl	65.0	8450	208500
Veenker	RL	Pave	FR2	Feedr	Norm	None	6	Lvl	Gtl	80.0	9600	181500
CollgCr	RL	Pave	Inside	Norm	Norm	None	7	Lvl	Gtl	68.0	11250	223500
Crawfor	RL	Pave	Corner	Norm	Norm	None	7	Lvl	Gtl	60.0	9550	140000
NoRidge	RL	Pave	FR2	Norm	Norm	None	8	Lvl	Gtl	84.0	14260	250000
Mitchel	RL	Pave	Inside	Norm	Norm	None	5	Lvl	Gtl	85.0	14115	143000
Somerst	RL	Pave	Inside	Norm	Norm	None	8	Lvl	Gtl	75.0	10084	307000
NWAmes	RL	Pave	Corner	PosN	Norm	None	7	Lvl	Gtl	69.0	10382	200000
OldTown	RM	Pave	Inside	Artery	Norm	None	7	Lvl	Gtl	51.0	6120	129900
BrkSide	RL	Pave	Corner	Artery	Artery	None	5	Lvl	Gtl	50.0	7420	118000
Sawyer	RL	Pave	Inside	Norm	Norm	None	5	Lvl	Gtl	70.0	11200	129500
NridgHt	RL	Pave	Inside	Norm	Norm	None	9	Lvl	Gtl	85.0	11924	345000
Sawyer	RL	Pave	Inside	Norm	Norm	None	5	Lvl	Gtl	69.0	12968	144000
CollgCr	RL	Pave	Inside	Norm	Norm	None	7	Lvl	Gtl	91.0	10652	279500
NAmes	RL	Pave	Corner	Norm	Norm	None	6	Lvl	Gtl	69.0	10920	157000
BrkSide	RM	Pave	Corner	Norm	Norm	None	7	Lvl	Gtl	51.0	6120	132000
NAmes	RL	Pave	CulDSac	Norm	Norm	None	6	Lvl	Gtl	69.0	11241	149000
Sawyer	RL	Pave	Inside	Norm	Norm	None	4	Lvl	Gtl	72.0	10791	90000
SawyerW	RL	Pave	Inside	RRRae	Norm	None	5	Lvl	Gtl	66.0	13695	159000
NAmes	RL	Pave	Inside	Norm	Norm	None	5	Lvl	Gtl	70.0	7560	139000
NridgHt	RL	Pave	Corner	Norm	Norm	None	8	Lvl	Gtl	101.0	14215	325300
IDOTRR	RM	Pave	Inside	Norm	Norm	Gvl	7	Bnk	Gtl	57.0	7449	139400

2.4 Eksplorasi Data Analisis (EDA) Lanjutan

a. Seluruh Fitur

EDA lanjutan dilakukan guna mengetahui penentuan model yang akan dibangun dengan tujuan memprediksi harga rumah dengan mempertimbangkan seluruh fitur yang ada. Berikut beberapa EDA lanjutanc yang dilakuka pada seluruh fitur pada dataset:

- Melihat struktur data setelah cleaning

```

=== SHAPE ===
(1460, 289)

=== INFO ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Columns: 289 entries, Id to SalePrice
dtypes: float64(288), int64(1)
memory usage: 3.2 MB
None

```

Dataset hasil preprocessing memiliki ukuran 1460 baris dan 289 kolom, yang menunjukkan bahwa terdapat 1460 observasi rumah dengan 288 fitur prediktor dan 1 variabel target yaitu SalePrice. Jumlah fitur yang besar ini berasal dari proses encoding variabel kategorikal menjadi numerik, sehingga informasi yang digunakan dalam pemodelan menjadi lebih lengkap dan detail. Seluruh fitur dalam dataset telah bertipe numerik, terdiri dari 288 kolom bertipe float64 dan 1 kolom bertipe int64, sehingga telah siap digunakan untuk proses pemodelan machine learning tanpa memerlukan konversi tipe data tambahan. Penggunaan memori yang relatif kecil (~3.2 MB) juga menunjukkan bahwa dataset telah terstruktur dengan efisien.

- Missing values dan Duplikat Data

```

=== CEK MISSING VALUES ===
0 missing values

=== CEK DUPLIKAT ===
Jumlah duplikat: 0

```

Hasil pengecekan menunjukkan bahwa tidak terdapat nilai kosong (missing value) pada seluruh fitur, yang menandakan bahwa proses imputasi berhasil dilakukan secara sempurna. Kondisi ini sangat baik untuk proses pemodelan karena model tidak akan terganggu oleh ketidaksempurnaan data akibat nilai yang hilang.

- Deskriptif statistik

EDA selanjutnya sangat penting untuk langkah memprediksi harga rumah yaitu menampilkan Statistik Deskriptif. Untuk menampilkan statistik deskriptif dapat menjalankan script berikut:

```

print("\n DESCRIPTIVE STATISTICS")
print(df.describe(include="all"))

```



```

... count OverallCond YearBuilt YearRemodAdd MasVnrArea BsmtFinSF1 \
mean 3.370211e-16 1.034180e-15 4.499293e-15 -4.866731e-17 7.300097e-18
std 1.000343e+00 1.000343e+00 1.000343e+00 1.000343e+00 1.000343e+00
min -4.112970e+00 -3.287824e+00 -1.689368e+00 -5.707501e-01 -9.730182e-01
25% -5.171998e-01 -5.719226e-01 -8.656586e-01 -5.707501e-01 -9.730182e-01
50% -5.171998e-01 5.737148e-02 4.425864e-01 -5.707501e-01 -1.319022e-01
75% 3.817427e-01 9.516316e-01 9.271216e-01 3.383686e-01 5.891327e-01
max 3.078570e+00 1.282839e+00 1.217843e+00 8.285201e+00 1.140575e+01

count ... SaleType_New SaleType_0th SaleType_WD SaleCondition_Abnorml \
mean ... 0.083562 0.002055 0.867808 0.069178
std ... 0.276824 0.045299 0.338815 0.253844
min ... 0.000000 0.000000 0.000000 0.000000
25% ... 0.000000 0.000000 1.000000 0.000000
50% ... 0.000000 0.000000 1.000000 0.000000
75% ... 0.000000 0.000000 1.000000 0.000000
max ... 1.000000 1.000000 1.000000 1.000000

count SaleCondition_AdjLand SaleCondition_Alloca SaleCondition_Family \
mean 0.002740 0.008219 0.013699
std 0.052289 0.090317 0.116277
min 0.000000 0.000000 0.000000
25% 0.000000 0.000000 0.000000
50% 0.000000 0.000000 0.000000
75% 0.000000 0.000000 0.000000
max 1.000000 1.000000 1.000000

count SaleCondition_Normal SaleCondition_Partial SalePrice
mean 0.820548 0.085616 180921.195890
std 0.383862 0.279893 79442.502883
min 0.000000 0.000000 34900.000000
25% 1.000000 0.000000 129975.000000
50% 1.000000 0.000000 163000.000000
75% 1.000000 0.000000 214000.000000
max 1.000000 1.000000 755000.000000

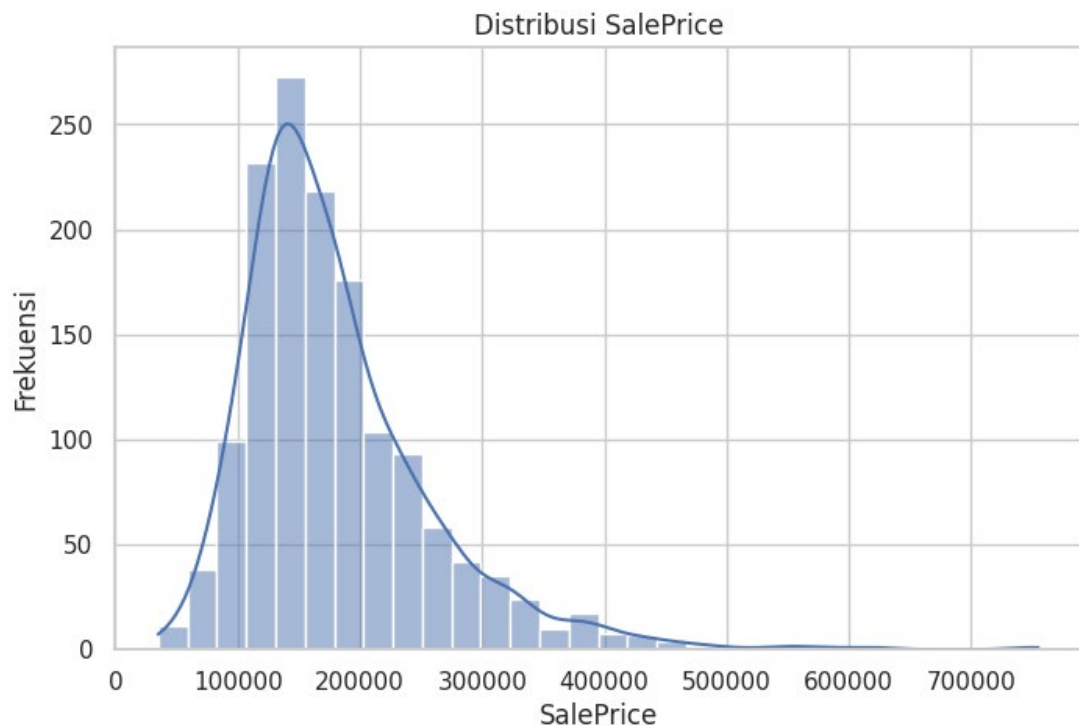
[8 rows x 289 columns]

```

Hasil statistik deskriptif menunjukkan bahwa nilai SalePrice memiliki rata-rata sebesar 180.921, dengan nilai minimum 34.900 dan maksimum 755.000, yang menggambarkan bahwa dataset mencakup rentang harga rumah dari kelas rendah hingga tinggi. Nilai standar deviasi yang besar menunjukkan variasi harga yang cukup tinggi, sehingga model diharapkan mampu menangkap pola perbedaan karakteristik rumah terhadap harga secara optimal.

- Visualisasi (Histogram Distribusi Data)
Membuat script code untuk menampilkan histogram semua fitur numerik di DataFrame dengan tujuan agar distribusinya mudah dibaca oleh orang awam. Berikut script yang dijalankan pada program:

```
plt.figure(figsize=(8, 5))
sns.histplot(df["SalePrice"], bins=30, kde=True)
plt.title("Distribusi SalePrice")
plt.xlabel("SalePrice")
plt.ylabel("Frekuensi")
plt.show()
```



Distribusi histogram SalePrice menunjukkan bahwa sebagian besar harga rumah terkonsentrasi pada rentang 100.000 hingga 200.000 dengan distribusi yang miring ke kanan (positively skewed), menandakan keberadaan sejumlah kecil rumah dengan harga sangat tinggi hingga di atas 500.000–700.000 yang berpotensi sebagai outlier. Namun, dalam penelitian ini penggunaan model berbasis pohon keputusan seperti Random Forest, XGBoost, dan LightGBM dianggap tetap sesuai tanpa perlu penghapusan outlier secara eksplisit, karena ketiga model tersebut tidak sensitif terhadap nilai ekstrem. Model tree-based membagi data berdasarkan batas nilai (split) sehingga outlier hanya memengaruhi sebagian kecil cabang pohon dan tidak mendistorsi keseluruhan proses pembelajaran seperti pada model regresi linear. Dengan demikian, pola sebaran harga yang tidak simetris ini tetap dapat dimanfaatkan secara optimal dalam proses pemodelan prediksi harga rumah.

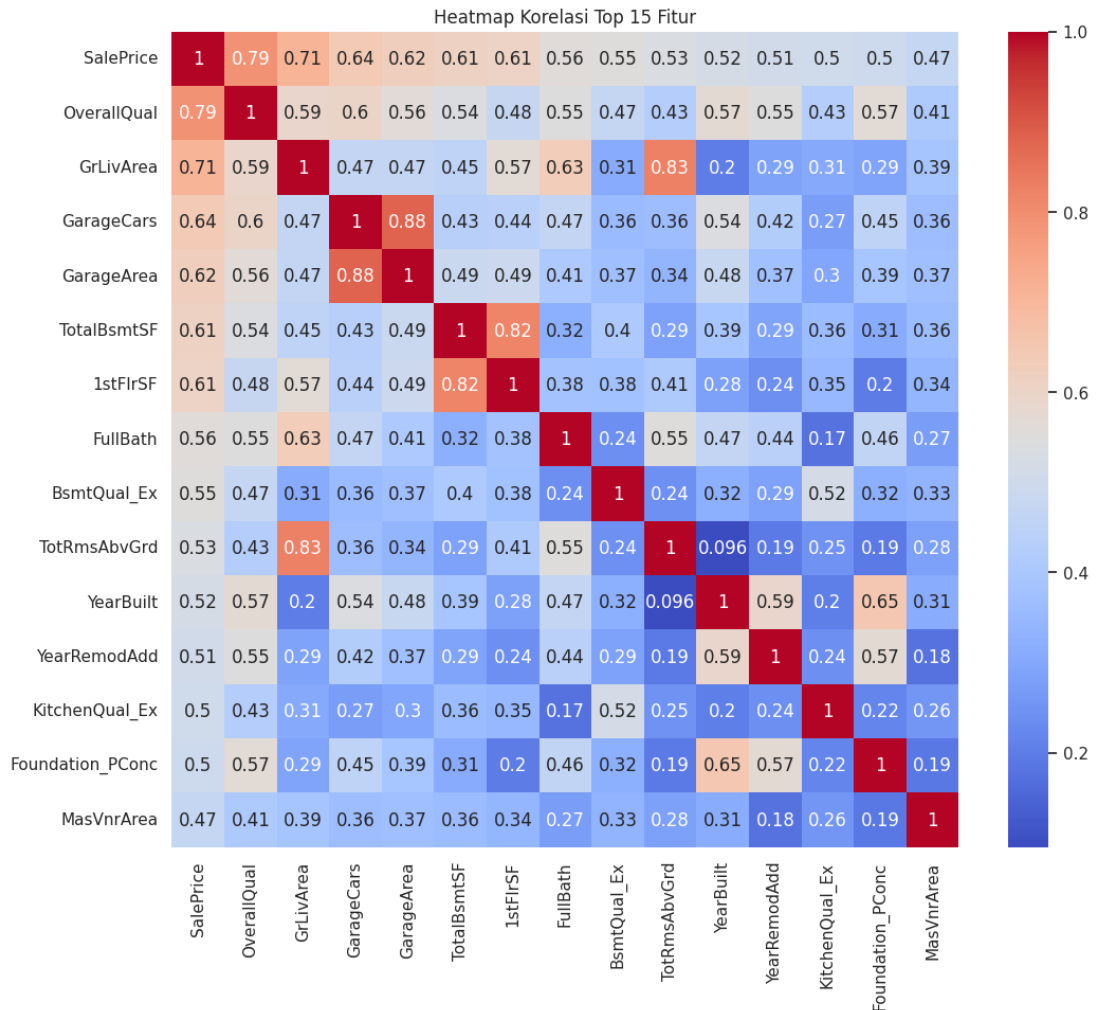
- Heatmap Korelasi

Membuat script code yang dijalankan pada program untuk menampilkan hubungan (korelasi) antar fitur numerik di dalam DataFrame df secara visual menggunakan

heatmap. Korelasinya dinyatakan dalam rentang +1,0 yang artinya hubungan kuat searah, -1,0 yang artinya hubungan kuat berlawanan, dan 0 yang artinya tidak ada hubungan. Berikut contoh code tersebut:

```
top_features = corr.head(15).index

plt.figure(figsize=(12, 10))
sns.heatmap(df[top_features].corr(), annot=True, cmap="coolwarm")
plt.title("Heatmap Korelasi Top 15 Fitur")
plt.show()
```



Heatmap korelasi menampilkan hubungan linear antara variabel SalePrice dengan 15 fitur yang memiliki korelasi tertinggi. Dari grafik terlihat bahwa korelasi antara fitur-fitur struktural rumah dengan harga sangat kuat, terutama fitur yang terkait kualitas bangunan dan luas area. Fitur dengan korelasi tertinggi terhadap SalePrice adalah OverallQual ($r = 0.79$), menunjukkan bahwa kualitas material dan pengerjaan rumah merupakan faktor paling dominan dalam menentukan harga. Fitur lainnya yang juga memiliki korelasi kuat adalah GrLivArea ($r = 0.71$), GarageCars ($r = 0.64$), GarageArea

($r = 0.62$), serta TotalBsmtSF ($r = 0.61$). Hal ini menggambarkan bahwa rumah dengan area permukaan luas, garasi besar, dan basement yang lebih besar cenderung memiliki harga lebih tinggi. Selain itu, fitur umur bangunan seperti YearBuilt ($r = 0.52$) dan YearRemodAdd ($r = 0.51$) juga berkorelasi cukup kuat, menunjukkan bahwa rumah yang lebih baru atau telah direnovasi cenderung lebih mahal. Secara keseluruhan, heatmap memperlihatkan pola korelasi positif kuat antar fitur yang berhubungan dengan ukuran dan kualitas konstruksi.

```
=== TOP 10 POSITIVE CORRELATION ===
SalePrice          1.000000
OverallQual        0.790982
GrLivArea          0.708624
GarageCars         0.640409
GarageArea         0.623431
TotalBsmtSF        0.613581
1stFlrSF           0.605852
FullBath           0.560664
BsmtQual_Ex        0.553105
TotRmsAbvGrd       0.533723
Name: SalePrice, dtype: float64

=== TOP 10 NEGATIVE CORRELATION ===
MSZoning_RM        -0.288065
MasVnrType_BrkFace -0.299378
HeatingQC_TA       -0.312677
BsmtExposure_No    -0.319990
Foundation_CBlock  -0.343263
GarageType_Detchd  -0.354141
BsmtQual_TA        -0.498545
GarageFinish_Unf    -0.513906
KitchenQual_TA     -0.519298
ExterQual_TA       -0.589044
Name: SalePrice, dtype: float64
```

Top 10 Positive Correlation merupakan daftar yang menegaskan kembali temuan dari heatmap, Fitur-fitur ini menunjukkan bahwa pembeli rumah cenderung memberikan

nilai premium pada kualitas konstruksi dan ukuran ruang hidup. Berikut fitur yang paling meningkatkan harga rumah:

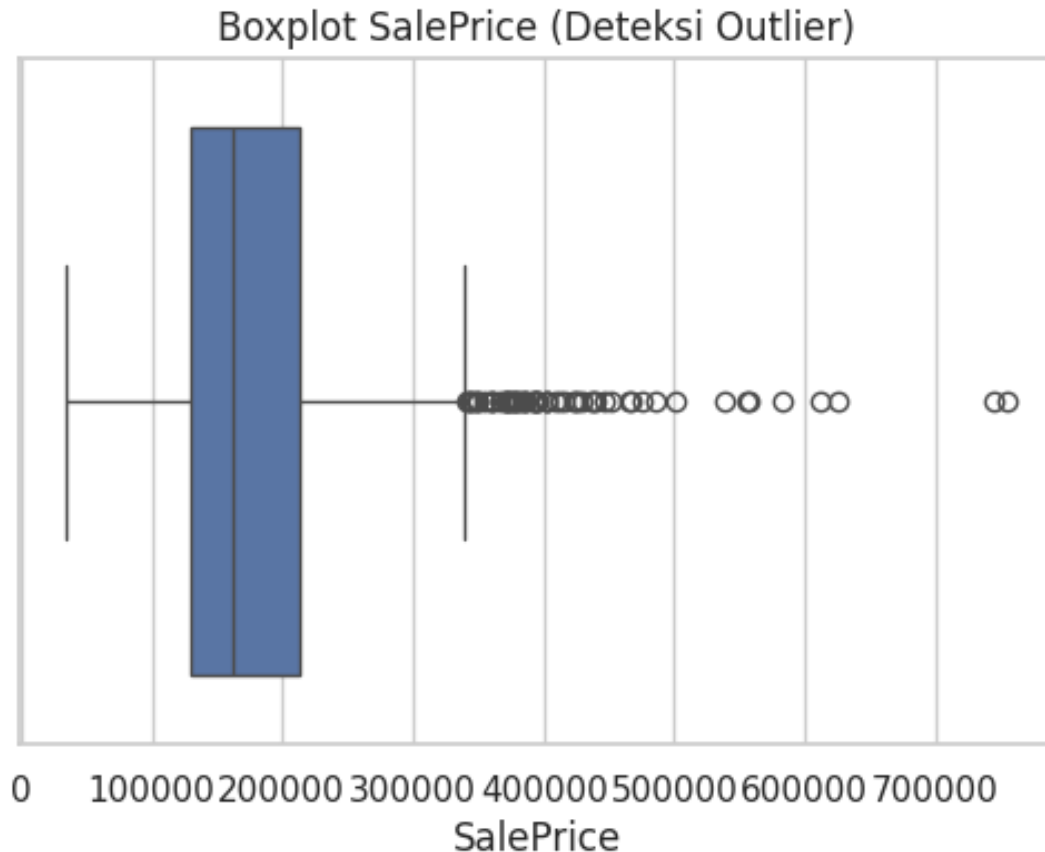
- OverallQual (0.79): kualitas keseluruhan rumah adalah prediktor paling kuat.
- GrLivArea (0.70): semakin luas area hunian, semakin mahal harga rumah.
- GarageCars (0.64) & GarageArea (0.62): kapasitas dan luas garasi berdampak signifikan pada harga.
- TotalBsmntSF (0.61) dan 1stFlrSF (0.60): menunjukkan bahwa semakin besar area total, basement, dan lantai pertama, semakin tinggi harga rumah.

Sedangkan Top 10 Negative Correlation merupakan daftar korelasi negatif menunjukkan fitur yang memiliki hubungan terbalik dengan harga rumah, meskipun kekuatannya moderat hingga lemah: MSZoning_RM (-0.28): rumah di zona RM (Residential Medium Density) cenderung lebih murah dibanding zona lain.

- MasVnrType_BrkFace (-0.29): rumah dengan veneer batu bata tipe BrkFace sedikit lebih murah dibanding tipe lain.
- HeatingQC_TA (-0.31): kualitas sistem pemanas "Typical/Average" memiliki kecenderungan harga lebih rendah.
- KitchenQual_TA (-0.51) dan ExterQual_TA (-0.58) menunjukkan bahwa kualitas dapur atau kualitas eksterior yang hanya "Average (TA)" berkorelasi kuat dengan harga yang lebih rendah.

Dengan demikian, daftar korelasi negatif memperlihatkan bahwa kualitas rata-rata (TA) pada beberapa aspek bangunan berasosiasi dengan harga yang lebih rendah, sedangkan fitur struktural dominan cenderung meningkatkan nilai rumah.

- Boxplot deteksi outlier



Boxplot variabel SalePrice menunjukkan bahwa sebagian besar harga rumah berada pada kisaran sekitar 120.000–250.000, dengan median di sekitar 180.000–190.000. Rentang antar kuartil (IQR) cukup lebar, menandakan variasi harga yang besar pada kelompok nilai tengah. Whisker bagian bawah berada di kisaran 80.000–90.000, sedangkan batas atas non-outlier berada sekitar 320.000–350.000. Posisi median yang lebih dekat ke kuartil bawah menunjukkan bahwa distribusi data bersifat right-skewed, atau condong ke kanan. Di luar batas whisker kanan, terlihat banyak titik outlier yang merupakan rumah dengan harga jauh lebih tinggi dari mayoritas, mulai dari sekitar 350.000 hingga mendekati 750.000. Banyaknya outlier di sisi kanan mengindikasikan keberadaan rumah berharga premium dalam jumlah kecil, sementara hampir tidak ada outlier di sisi rendah. Hal ini memperkuat bahwa distribusi SalePrice memiliki ekor panjang ke kanan dan didominasi oleh rumah dengan harga menengah, sedangkan rumah dengan harga sangat tinggi menjadi outlier yang signifikan dalam dataset.

b. 12 Fitur Berdasarkan Lokasi

Dilakukan juga EDA lanjutan pada dataset yang telah dilakukan filter pada 12 fitur berdasarkan lokasi untuk dilakukan prediksi nantinya. Berikut EDA yang dilakukan pada 12 fitur berdasarkan lokasi:

- Melihat Struktur Data setelah cleaning

```

--- SHAPE ---
(1460, 13)

--- INFO ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Neighborhood    1460 non-null   object
1   MSZoning        1460 non-null   object
2   Street          1460 non-null   object
3   LotConfig       1460 non-null   object
4   Condition1      1460 non-null   object
5   Condition2      1460 non-null   object
6   Alley           91 non-null     object
7   OverallQual     1460 non-null   int64
8   LandContour     1460 non-null   object
9   LandSlope       1460 non-null   object
10  LotFrontage     1460 non-null   float64
11  LotArea         1460 non-null   int64
12  SalePrice       1460 non-null   int64
dtypes: float64(1), int64(3), object(9)
memory usage: 148.4+ KB
None

```

Dari hasil cleaning yang menggunakan 12 label yaitu yang berkaitan dengan lokasi didapatkan bahwa dataset memiliki 1460 baris (observasi) dan memiliki 13 kolom (fitur) hal ini data train Ames Housing + SalePrice. Selanjutnya didapatkan tipe data yaitu: 9 kolom kategorikal (object) yaitu Neighborhood, MSZoning, Street, Lot Config, Condition 1, Condition 2, Alley, Land Contour, LandSlope, serta 3 kolom numerik integer yaitu OverallQual, LotArea, SalePrice, dan 1 kolom numerik float yaitu LotFrontage.

- Missing values

Pada proses cleaning juga dilakukan handling missing value pada kolom numerik dan kategorikal, Berikut script yang dijalankan untuk handling missing value pada dua tipe data yang berbeda:

```

print("\nCEK MISSING VALUES")
print(df.isnull().sum())

```

Kolom LotFrontage sering terjadi missing sekitar 17-18% pada dataset. Kolom LotFrontage yang missing akan diisi dengan nilai median sebab median tidak dipengaruhi outlier. Pada kolom kategorikal Alley yang memiliki missing >90% akan diisi oleh None. Sedangkan pada kolom kategori lainnya yang memiliki missing akan

diisi dengan Unknown dengan tujuan tidak boleh ada NaN sebelum dilakukan encoding. Dari script tersebut akan dihasilkan output berikut:

```
--- CEK MISSING VALUES ---  
Neighborhood      0  
MSZoning          0  
Street            0  
LotConfig         0  
Condition1        0  
Condition2        0  
Alley            1369  
OverallQual       0  
LandContour       0  
LandSlope         0  
LotFrontage       0  
LotArea           0  
SalePrice         0  
dtype: int64
```

Melihat dari hasil dapat diinterpretasikan semua kolom sudah berhasil melakukan handling missing value, namun masih terdapat 1369 yang belum berhasil di hapus pada kolom Alley. Dengan kata lain kolom alley yang masih memiliki nilai kosong yaitu 91 non-null dari 1460. Meski begitu code akan melakukan pengisian data pada variabel data sehingga dalam file cleaning kolom Alley sudah dipastikan tidak ada missingnya.

- Deskriptif statistik

EDA selanjutnya sangat penting untuk langkah memprediksi harga rumah yaitu menampilkan Statistik Deskriptif Untuk menampilkan statistik deskriptif dapat menjalankan script berikut:

```
print("\n DESCRIPTIVE STATISTICS")  
print(df.describe(include="all"))
```



```

--- DESCRIPTIVE STATISTICS ---
***
      Neighborhood MSZoning Street LotConfig Condition1 Condition2 Alley \
count      1460      1460      1460      1460      1460      1460      91
unique       25       5       2       5       9       8       2
top      NAmes      RL      Pave      Inside      Norm      Norm      GrvL
freq       225     1151     1454     1052     1260     1445     50
mean       NaN      NaN      NaN      NaN      NaN      NaN      NaN
std        NaN      NaN      NaN      NaN      NaN      NaN      NaN
min        NaN      NaN      NaN      NaN      NaN      NaN      NaN
25%        NaN      NaN      NaN      NaN      NaN      NaN      NaN
50%        NaN      NaN      NaN      NaN      NaN      NaN      NaN
75%        NaN      NaN      NaN      NaN      NaN      NaN      NaN
max        NaN      NaN      NaN      NaN      NaN      NaN      NaN

      OverallQual LandContour LandSlope LotFrontage      LotArea \
count  1460.000000      1460      1460  1460.000000  1460.000000
unique      NaN       4       3      NaN      NaN
top      NaN      Lvl      Gtl      NaN      NaN
freq      NaN     1311     1382      NaN      NaN
mean      6.099315      NaN      NaN    69.863699  10516.828082
std      1.382997      NaN      NaN    22.027677   9981.264932
min      1.000000      NaN      NaN    21.000000  1300.000000
25%      5.000000      NaN      NaN    60.000000  7553.500000
50%      6.000000      NaN      NaN    69.000000  9478.500000
75%      7.000000      NaN      NaN    79.000000  11601.500000
max     10.000000      NaN      NaN   313.000000 215245.000000

      SalePrice
count  1460.000000
unique      NaN
top      NaN
freq      NaN
mean   180921.195890
std    79442.502883
min    34900.000000
25%    129975.000000
50%    163000.000000
75%    214000.000000
max    755000.000000

```

Dari script tersebut akan menghasilkan output berupa rangkuman statistik dataset yang sudah di lakukan berbagai proses. Pada kolom kategorikal ada beberapa kolom yang sangat dominan dan adapula kategori yang jarang. Alley hampir selalu None yang artinya ini bersifat informatif tapi tidak cocok kalau dimasukkan sebagai banyak dummy kolom. Sedangkan pada kolom numerik banyak banyak fitur (LotArea, SalePrice, LotFrontage) punya rentang luas dan outlier. Berikut penjelasan secara rincinya:

- o Label kategorik:

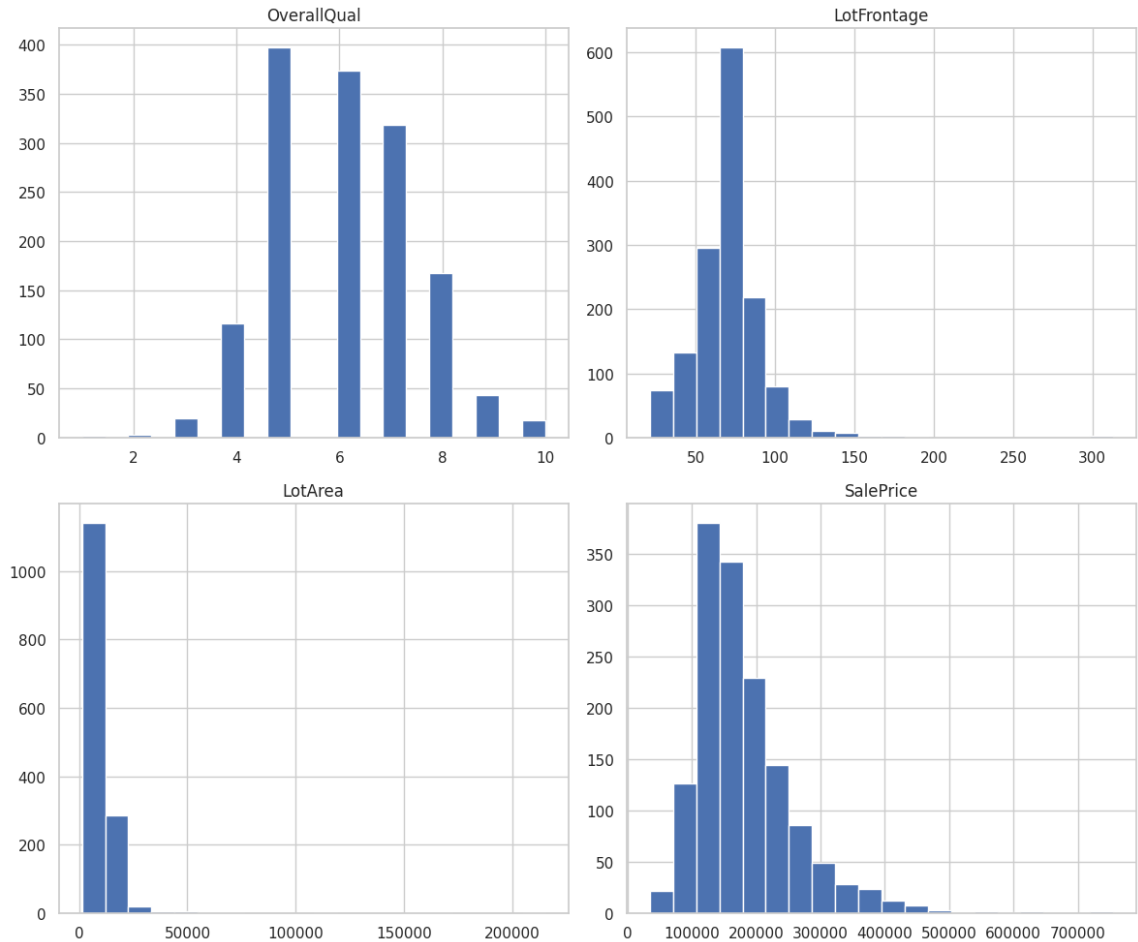
1. Neighborhood (lokasi/wilayah rumah) dimana memiliki interpretasi kota/area sampel cukup beragam (25 wilayah). NAmes menjadi area yang paling padat/banyak rumahnya.
2. MSZoning dimana memiliki interpretasi mayoritas besar rumah berada di zona RL (Residential Low Density) dan artinya rumah-rumah ini berada di lingkungan perumahan dengan kepadatan rendah.
3. Street dimana memiliki interpretasi hampir semua rumah berada di jalan beraspal (Pave) dan sedikit yang berada di jalan kerikil (Gravel).

4. LotConfig memiliki interpretasi mayoritas rumah berada di posisi Inside (bukan di tikungan atau cul-de-sac) dan distribusi lahan cukup tidak merata karena satu kategori sangat dominan.
 5. Condition1 memiliki interpretasi 86% rumah memiliki kondisi lingkungan yang normal, tidak dekat rel kereta atau jalan besar.
 6. Condition2 memiliki interpretasi hampir semua rumah juga normal pada kondisi kedua (jarang terjadi efek lain seperti rel/jalan tambahan).
 7. Alley memiliki jarang sekali rumah memiliki akses gang, sehingga kolom ini wajar diberi kategori 'None' untuk missing dan rumah dengan gang gravel sedikit lebih banyak daripada yang paved.
 8. LandContour memiliki interpretasi hampir semua rumah berada di kontur tanah datar (Level) dan rumah di tanah miring atau berundak sangat jarang.
 9. LandSlope memiliki interpretasi mayoritas tanah memiliki kemiringan lembut/landai (Gentle Slope) dan tanah curam sangat jarang.
- Label numerik:
 1. OverallQual memiliki interpretasi rumah yang dijual umumnya punya kualitas baik, tidak terlalu rendah dan tidak terlalu tinggi.
 2. LotFrontage memiliki interpretasi sebagian besar rumah punya lebar lahan normal (60–80 kaki), tetapi ada properti yang sangat luas yang dapat memengaruhi model.
 3. LotArea memiliki interpretasi mayoritas rumah punya lahan sedang, namun terdapat beberapa rumah dengan lahan super besar yang tidak umum.
 4. Saleprice memiliki interpretasi Harga rata-rata rumah sekitar 181 ribu USD. Namun terdapat sejumlah kecil rumah sangat mahal yang jauh di atas harga rata-rata yang artinya outlier high-end houses.
 - Visualisasi (Histogram Distribusi Data)

Membuat script code untuk menampilkan histogram semua fitur numerik di DataFrame dengan tujuan agar distribusinya mudah dibaca oleh orang awam. Berikut script yang dijalankan pada program:

```
df.hist(figsize=(12, 10), bins=20)
plt.tight_layout()
plt.show()
```

Code tersebut akan mencari semua kolom numerik lalu membuat histogram untuk setiap kolom numerik tersebut. Histogram akan ditampilkan semua dalam satu grid. Tujuan dari membuat histogram tersebut untuk mengetahui apakah distribusi datanya normal, skew, atau punya outlier. Histogram juga dapat melihat pola penyebaran nilai untuk selanjutnya menentukan tindakan apakah perlu dilakukan scaling atau transformasi. Berikut histogram yang terbentuk:



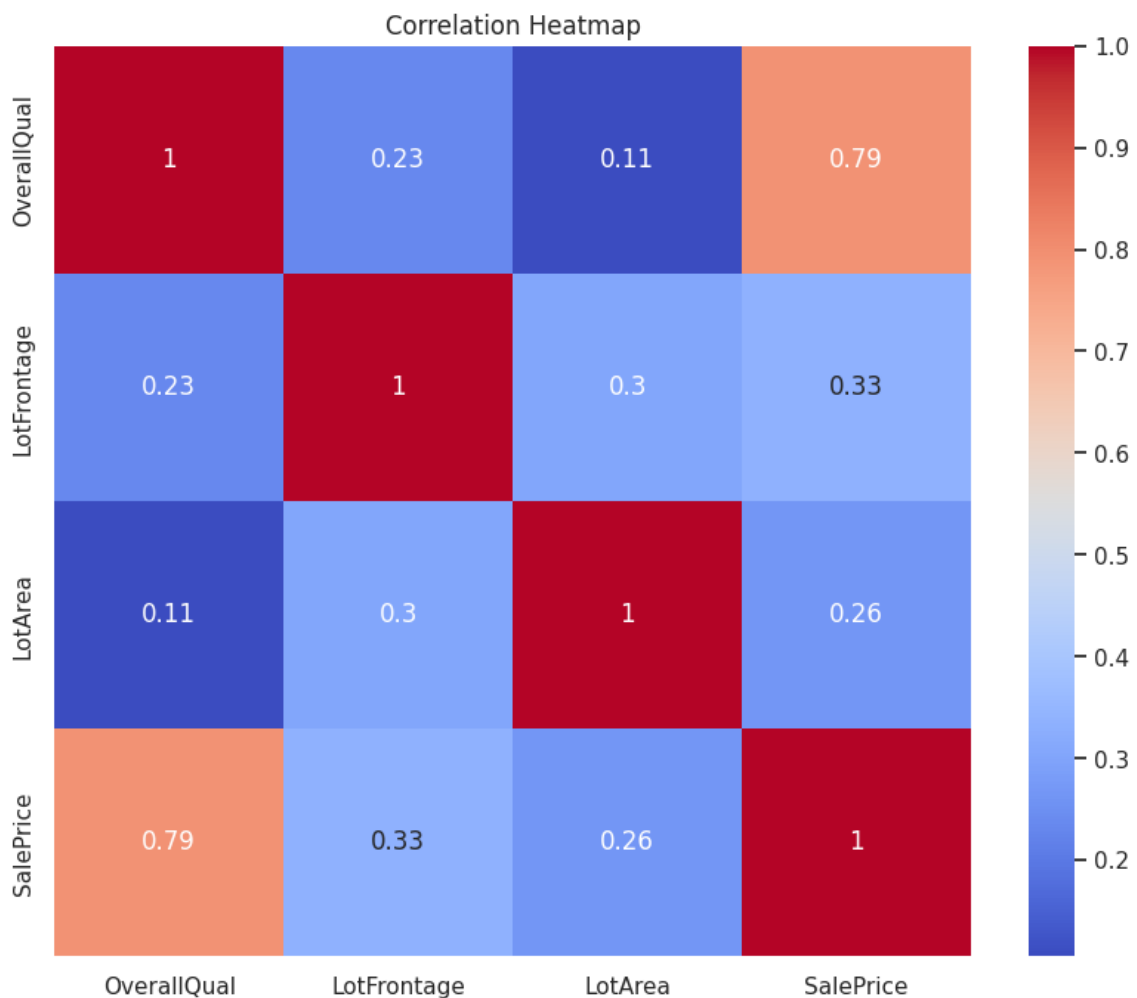
Dari histogram tersebut dapat dijelaskan secara detail sebagai berikut:

- OverallQual
 1. Distribusi terlihat cenderung normal (mendekati bell-curve).
 2. Nilai 5–7 adalah yang paling banyak sehingga mayoritas rumah memiliki kualitas sedang.
 3. Semakin tinggi nilai ini maka harga rumah biasanya semakin tinggi.
- LotFrontage
 1. Distribusi condong ke kiri (right skewed).
 2. Banyak rumah memiliki frontage sekitar 50–80 kaki.
 3. Ada beberapa outlier di atas 150–200.
- LotArea
 1. Distribusi sangat right skewed.
 2. Banyak rumah memiliki area kecil (5000–10000), tapi ada sedikit rumah dengan area sangat besar (hingga ratusan ribu) dan ini adalah outlier.
- Saleprice
 1. Distribusi right skewed.
 2. Harga rumah paling banyak berkisar di 100k–200k.
 3. Ada beberapa rumah mahal (hingga 700k) sehingga outlier.

- Heatmap Korelasi

Membuat script code yang dijalankan pada program untuk menampilkan hubungan (korelasi) antar fitur numerik di dalam DataFrame df secara visual menggunakan heatmap. Korelasinya dinyatakan dalam rentang +1,0 yang artinya hubungan kuat searah, -1,0 yang artinya hubungan kuat berlawanan, dan 0 yang artinya tidak ada hubungan. Berikut contoh code tersebut:

```
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



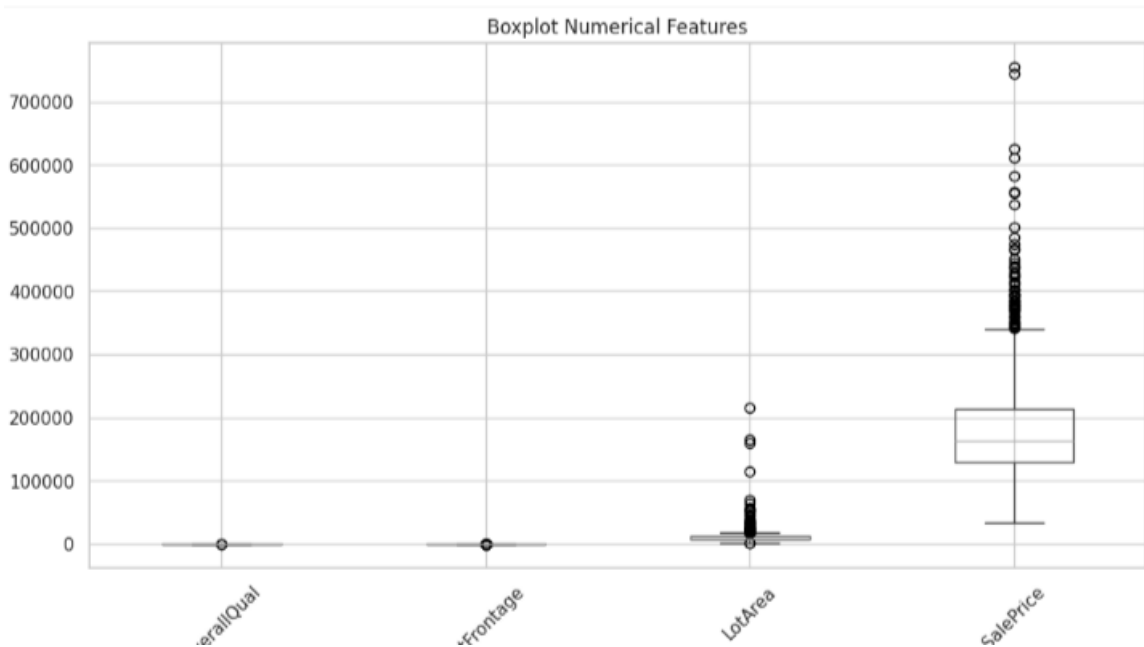
Script tersebut akan menghasilkan output yang dapat diinterpretasikan sebagai berikut:

- OverallQual ke SalePrice = 0.79, memiliki korelasi sangat kuat. Semakin bagus kualitas rumah, harga makin tinggi.
- LotFrontage ke SalePrice = 0.33, memiliki korelasi lemah-menengah. Lebar depan rumah memengaruhi harga, tapi tidak dominan.

- LotArea ke SalePrice = 0.26, memiliki korelasi lemah. Ukuran tanah tidak terlalu menentukan harga.
- OverallQual adalah fitur paling penting. LotFrontage & LotArea memberikan kontribusi kecil terhadap prediksi harga. Tidak ada multicollinearity parah antara ketiga numerik ini.
- Boxplot deteksi outlier

Membuat boxplot yang menampilkan semua kolom numerik di DataFrame df dengan tujuan untuk mendeteksi outlier dan melihat sebaran data numerik dengan cepat. Berikut script yang dijalankan pada program:

```
plt.figure(figsize=(12, 6))
df.select_dtypes(include=["int64", "float64"]).boxplot()
plt.title("Boxplot Numerical Features")
plt.xticks(rotation=45)
plt.show()
```



Berdasarkan boxplot fitur numerik, terdapat beberapa outlier yang cukup ekstrem terutama pada variabel LotArea dan SalePrice, yang menunjukkan adanya rumah dengan luas tanah dan harga jauh di atas mayoritas sampel. Hal ini perlu dipertimbangkan dalam tahap pemodelan karena berpotensi mempengaruhi performa model.

2.5 Penggunaan Algoritma

Dalam memprediksi harga rumah kami menggunakan dua algoritma berbeda dengan data yang sama untuk dapat dilihat perbandingannya dan menemukan algoritma terbaik untuk

melakukan prediksi harga rumah. Project ini kami menggunakan algoritma inti Random Forest dan XGBoost, juga menambahkan algoritma gabungan XGBoost + LightGBM.

a. Seluruh Fitur

- Random forest

```
rf_model = RandomForestRegressor(  
    n_estimators=300,  
    max_depth=None,  
    random_state=42,  
    n_jobs=-1  
)  
  
rf_model.fit(X_train_processed, y_train)
```

Model membangun 300 pohon, sehingga hasil prediksinya lebih kuat karena setiap pohon memberikan kontribusi. Parameter `max_depth=None` menunjukkan bahwa kedalaman setiap pohon tidak dibatasi, sehingga pohon akan tumbuh maksimal sesuai pola data. Pengaturan `random_state=42` bertujuan agar hasil model selalu sama ketika dijalankan kembali, karena proses random diatur dengan angka seed tersebut. Parameter `n_jobs=-1` membuat proses pelatihan memanfaatkan seluruh inti CPU agar training berjalan lebih cepat. Setelah model selesai dibuat, perintah `rf_model.fit(X_train_processed, y_train)` menjalankan proses pembelajaran, yaitu model membaca pola dari fitur yang sudah diproses (`X_train_processed`) dan mempelajari hubungan dengan nilai target (`y_train`).

```
===== Random Forest =====  
MAE : 17467.572157534247  
RMSE: 28572.848758496166  
R2 : 0.8935628467260077
```

Model Random Forest menghasilkan MAE sebesar sekitar 17.467, yang berarti rata-rata kesalahan prediksi harga berada di kisaran 17 ribu satuan harga dari nilai sebenarnya. RMSE sebesar 28.572 menunjukkan bahwa kesalahan yang lebih besar masih terjadi, karena RMSE lebih sensitif terhadap error ekstrem. Nilai R^2 sebesar 0.8935 menandakan bahwa model mampu menjelaskan sekitar 89,35% variasi pada data target, yang tergolong sangat baik. Melihat nilai tersebut, dapat dilakukan Hyperparameter Tuning untuk mendapatkan akurasi yang lebih tinggi, berikut script yang digunakan untuk melakukan Hyperparameter Tuning:

```

rf_param_dist = {
    "n_estimators": randint(200, 1000),
    "max_depth": randint(5, 50),
    "min_samples_split": randint(2, 20),
    "min_samples_leaf": randint(1, 10),
    "max_features": ["sqrt", "log2"]
}
rf_random_search = RandomizedSearchCV(
    estimator=RandomForestRegressor(random_state=42),
    param_distributions=rf_param_dist,
    n_iter=30,
    scoring="neg_root_mean_squared_error",
    cv=3,
    verbose=2,
    n_jobs=-1,
    random_state=42
)
rf_random_search.fit(X_train_processed, y_train)

```

Kode tersebut digunakan untuk menemukan pengaturan terbaik bagi model Random Forest secara otomatis. RandomizedSearchCV akan mengambil 30 kombinasi acak dari parameter-parameter tersebut dan menguji setiap kombinasi menggunakan 3-fold cross-validation untuk melihat mana yang menghasilkan performa terbaik. Penilaian menggunakan RMSE sehingga model terbaik adalah model dengan error paling kecil. Pengaturan `n_jobs=-1` membuat proses berjalan lebih cepat dengan memanfaatkan semua inti CPU. Terakhir, perintah `fit()` menjalankan seluruh proses pencarian sehingga Random Forest dapat menemukan kombinasi hyperparameter yang paling optimal.

```

===== Random Forest (Tuned) =====
MAE : 18857.728181724535
RMSE: 33925.66102312457
R2 : 0.8499476495826854

```

- XGBoost

```

xgb_model = XGBRegressor(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    tree_method="hist"
)

xgb_model.fit(X_train_processed, y_train)

```

Kode tersebut membangun model XGBoost Regressor dengan 500 pohon dan laju belajar kecil (0.05) agar proses pembelajaran lebih stabil, menggunakan pohon dengan

kedalaman maksimal 6 serta hanya mengambil 80% data dan 80% fitur di setiap pohon untuk mencegah overfitting. Parameter `tree_method="hist"` digunakan agar proses training lebih cepat dan efisien, sementara `random_state=42` memastikan hasil konsisten. Setelah konfigurasi, perintah `xgb_model.fit(X_train_processed, y_train)` melatih model dengan data training yang sudah diproses sehingga model dapat mempelajari pola dan hubungan antara fitur dan target.

```
===== XGBoost =====  
MAE : 15685.7587890625  
RMSE: 25721.255334839316  
R2 : 0.9137477278709412
```

Model XGBoost menghasilkan MAE sekitar 15.685, yang lebih rendah dibandingkan Random Forest, menunjukkan prediksi rata-rata lebih akurat. RMSE sebesar 25.721 juga lebih kecil, menandakan XGBoost lebih baik dalam mengatasi error besar. Nilai R^2 sebesar 0.9137 menunjukkan bahwa model ini mampu menjelaskan 91,37% variasi pada data, sehingga performa keseluruhan lebih kuat daripada Random Forest. Secara umum, XGBoost memberikan prediksi lebih presisi dan stabil. Dengan nilai tersebut, dapat dilakukan Hyperparameter Tuning untuk mendapatkan akurasi yang lebih tinggi:

```
xgb_param_dist = {  
    "n_estimators": randint(300, 1200),  
    "learning_rate": uniform(0.01, 0.2),  
    "max_depth": randint(3, 15),  
    "subsample": uniform(0.6, 0.4),  
    "colsample_bytree": uniform(0.6, 0.4)  
}  
  
xgb_random_search = RandomizedSearchCV(  
    estimator=XGBRegressor(  
        objective="reg:squarederror",  
        random_state=42,  
        tree_method="hist"  
    ),  
    param_distributions=xgb_param_dist,  
    n_iter=30,  
    scoring="neg_root_mean_squared_error",  
    cv=3,  
    verbose=2,  
    n_jobs=-1,  
    random_state=42  
)  
  
xgb_random_search.fit(X_train_processed, y_train)
```

Kode tersebut digunakan untuk mencari kombinasi hyperparameter terbaik pada model XGBoost secara otomatis. `RandomizedSearchCV` akan mencoba 30 kombinasi acak dari parameter tersebut dan mengevaluasi masing-masing menggunakan 3-fold cross-validation untuk menentukan mana yang menghasilkan RMSE paling rendah. Model dasar yang diuji adalah `XGBRegressor` dengan objective regresi dan metode pohon "hist" yang lebih cepat. Pengaturan `n_jobs=-1` mempercepat proses dengan memakai semua inti CPU, sedangkan `verbose=2` menampilkan proses pencarian secara detail.

Terakhir, perintah `fit()` menjalankan seluruh proses sehingga menghasilkan konfigurasi XGBoost yang paling optimal berdasarkan performa terbaik.

```
===== XGBoost (Tuned) =====  
MAE : 15519.8955078125  
RMSE: 24443.396163381225  
R2 : 0.922105073928833
```

Hasil tuning menunjukkan dampak yang berbeda pada kedua model. Pada Random Forest, performa justru menurun setelah proses pencarian hyperparameter. MAE meningkat menjadi sekitar 18.857, RMSE melonjak hingga 33.925, dan nilai R^2 turun menjadi 0.8499, sehingga model menjadi kurang akurat dan kurang mampu menjelaskan variasi data dibandingkan versi sebelum tuning. Hal ini menunjukkan bahwa kombinasi parameter hasil `RandomizedSearch` tidak cocok untuk dataset ini. Sebaliknya, tuning pada XGBoost menghasilkan peningkatan performa yang jelas. MAE turun menjadi sekitar 15.519, RMSE membaik menjadi 24.443, dan R^2 meningkat hingga 0.9221, menandakan model semakin presisi dan mampu menjelaskan lebih banyak variasi target. Secara keseluruhan, tuning memberikan efek negatif pada Random Forest tetapi memberikan hasil optimal dan meningkatkan akurasi pada XGBoost.

- LGBM dan XGBoost

```

xgb = XGBRegressor(
    n_estimators=1200,
    learning_rate=0.03,
    max_depth=5,
    subsample=0.9,
    colsample_bytree=0.9,
    reg_lambda=1.2,
    reg_alpha=0.1,
    tree_method="hist",
    random_state=42
)

lgb = LGBMRegressor(
    n_estimators=1600,
    learning_rate=0.03,
    num_leaves=50,
    subsample=0.9,
    colsample_bytree=0.9,
    min_child_samples=20,
    random_state=42
)

ridge = Ridge(
    alpha=1.0,
    random_state=42
)

```

Kode tersebut melatih model XGBoost dan LightGBM dalam memiliki konfigurasi hyperparameter yang dirancang untuk menghasilkan pembelajaran yang stabil, kuat, dan mampu menangkap pola data yang kompleks, sehingga sangat mendukung performa tinggi pada konfigurasi fitur yang lebih banyak. XGBoost memanfaatkan *learning rate* kecil (0.03) dengan jumlah pohon yang besar (1200) serta kedalaman pohon yang moderat, sehingga mampu mempelajari pola non-linear secara bertahap tanpa mudah overfitting, diperkuat oleh mekanisme *subsample*, *colsample_bytree*, dan regularisasi L1–L2. LightGBM menggunakan prinsip yang serupa dengan *learning rate* kecil, jumlah pohon sangat banyak (1600), serta *num_leaves* yang cukup besar untuk menangkap interaksi fitur, namun tetap dikendalikan oleh *min_child_samples* agar tidak melakukan split yang tidak signifikan. Sementara itu, Ridge Regression berperan sebagai model linear reguler yang memberikan baseline stabil melalui regularisasi L2, sehingga membantu mengurangi sensitivitas terhadap multikolinearitas.

```

===== FINAL ENSEMBLE RESULT =====
MAE : 15105.45803195204
RMSE: 25267.323700695506
R2 : 0.9167652548600201

```

Hasil akhir model ensemble menunjukkan performa prediksi yang sangat baik dengan nilai MAE sebesar 15.105, RMSE sebesar 25.267, serta nilai R^2 sebesar 0.916. Nilai MAE dan RMSE yang relatif rendah mengindikasikan bahwa kesalahan rata-rata prediksi model berada pada tingkat yang dapat diterima dan model mampu menjaga stabilitas prediksinya meskipun dihadapkan pada nilai target yang bervariasi. Sementara itu, nilai R^2 sebesar 0.916 menegaskan bahwa model ensemble mampu menjelaskan sekitar 91,6% variansi data, yang menunjukkan kemampuan generalisasi yang kuat serta kecocokan yang sangat baik antara model dan pola yang terdapat dalam dataset. Performa ini mengonfirmasi bahwa penggabungan beberapa model—khususnya model berbasis boosting seperti XGBoost dan LightGBM—memberikan kontribusi signifikan terhadap peningkatan akurasi.

b. 12 Fitur Berdasarkan Lokasi

- Algoritma Random Forest

Dengan menggunakan dataset hasil cleaning dilakukan splitting dengan membagi 80% data training dan 20% data testing. Menentukan nilai X (berisi fitur) dan Y (berisi target). Script juga akan mendeteksi otomatis kolom kategorik dan numerik untuk kepentingan pipeline.

```
categorical_cols = X_train.select_dtypes(include=["object"]).columns.tolist()
numeric_cols = X_train.select_dtypes(exclude=["object"]).columns.tolist()
```

Setelahnya membentuk kolom transformer, dengan dua processing. Kolom numerik menggunakan Standar Scaler dengan mengubah skala mean = 0 dan std = 1. Standar Scaler juga akan membuat model lebih stabil dengan menangani outlier lebih baik dalam model linear/NN. Sedangkan pada kolom kategori menggunakan OneHotEncoder dengan mengubah value kategori menjadi kolom dummy 0/1. Satu hal yang diperhatikan, kategori baru pada data input tidak bikin error.

```
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), categorical_cols),
])
```

Langkah penting sebelum melakukan train model dan mendapatkan prediksi harganya, perlu dibuat full pipeline yang memadukan preprocessing dan model. Script langsung melakukan OHE + scaling + training model hanya dengan `.fit()` yang akan menjamin preprocessing di train dan test konsisten. Berikut script yang digunakan dan output yang dikeluarkan oleh program:

```
rf_model = Pipeline([
    ("preproc", preprocessor),
    ("model", RandomForestRegressor(
        n_estimators=300,
        random_state=42,
        n_jobs=-1
    ))
])
```

```
MAE : 24515.09760935982
RMSE: 37781.5648979894
R2   : 0.8139001383274336
```

Melihat tiga aspek penting untuk mengetahui apakah prediksi memiliki akurasi yang baik masih memiliki angka yang tinggi, maka dilakukan Hyperparameter Tuning untuk menurunkan nilai ketiga aspek tersebut agar akurasi prediksi semakin baik. Berikut script yang digunakan untuk melakukan Hyperparameter Tuning pada algoritma Random Forest:

```
rf = RandomForestRegressor(random_state=42)
rf_param = {
    "n_estimators": randint(200, 800),
    "max_depth": randint(5, 40),
    "min_samples_split": randint(2, 20),
    "min_samples_leaf": randint(1, 10)
}
rf_search = RandomizedSearchCV(rf, rf_param, n_iter=20, cv=3, scoring="neg_mean_absolute_error", n_jobs=-1)
rf_search.fit(X_train, y_train)
rf_pred = rf_search.best_estimator_.predict(X_test)
```

Proses tuning model Random Forest dilakukan menggunakan *RandomizedSearchCV* untuk mendapatkan kombinasi hyperparameter terbaik secara efisien. Rentang nilai seperti jumlah pohon (*n_estimators*), kedalaman maksimum pohon (*max_depth*), serta parameter pembagian node (*min_samples_split* dan *min_samples_leaf*) ditentukan sebagai ruang pencarian. *RandomizedSearchCV* kemudian mencoba 20 kombinasi acak dengan *3-fold cross-validation* dan menilai performa setiap kombinasi menggunakan metrik *negative mean absolute error*. Setelah proses pencarian selesai, model dengan kinerja terbaik dipilih melalui atribut *best_estimator_* dan digunakan untuk melakukan prediksi pada data uji. Pendekatan ini memungkinkan peningkatan akurasi model dengan waktu komputasi yang lebih cepat dibandingkan metode *grid search* penuh. Dengan Hyperparameter Tuning akan dihasilkan nilai sebagai berikut:

```
=== RandomForest ===  
MAE : 24364.07636245851  
RMSE: 38676.64613536224  
R2 : 0.8049779219456947
```

- Algoritma XGBoost

Secara umum, langkah-langkah yang dijalankan pada algoritma XGBoost sama seperti algoritma Random Forest. Data yang di load merupakan dataset hasil cleaning yang belum dilakukan one hot encoding sebab nanti akan dilakukan preprocessing pipeline pada algoritma XGBoost yang dijalankan. Terdapat perbedaan pada pembentukan preprocessing pipeline, algoritma XGBoost tidak mewajibkan scaling, namun scaling tetap membantu training stabil.

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ("num", StandardScaler(), numeric_cols),  
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),  
    ]  
)
```

Setelah preprocessing pipeline terbentuk, dilanjut untuk membangun model dengan parameter penting seperti jumlah pohon, kedalaman pohon, dan sebagian fitur yang dipakai setiap pohonnya. Selanjutnya, membentuk full pipeline yang memadukan preprocessing dan model dengan keunggulan menjamin train dan test di preprocessing dengan cara yang sama.

```
pipeline = Pipeline(steps=[  
    ("preprocessor", preprocessor),  
    ("model", xgb_model)  
])
```

Full pipeline digunakan untuk melatih model dengan bekerja sebagai fit scaler di numerik, fit one-hot encoder di kategorikal, transformasi input, dan melakukan train XGBoost. Model yang dilatih akan mendapatkan harga prediksi rumah dari dataset testing. Berikut output yang dikeluarkan dari membangun algoritma XGBoost:

```
===== XGBoost Regression Results =====  
MAE : 24136.990234375  
RMSE : 36465.26895554179  
R2 : 0.8266415596008301
```

Melihat hasil dari algoritma XGBoost tersebut dapat dilakukan Hyperparameter Tuning untuk menurunkan nilai dari ketiga aspek tersebut dan mendapatkan akurasi prediksi

yang lebih tinggi. Berikut script yang digunakan untuk melakukan Hyperparameter Tuning pada algoritma XGBoost:

```
xgb = XGBRegressor(random_state=42, tree_method="hist")
xgb_param = {
    "n_estimators": randint(300, 1000),
    "learning_rate": uniform(0.01, 0.2),
    "max_depth": randint(3, 12),
    "subsample": uniform(0.5, 0.5),
    "colsample_bytree": uniform(0.5, 0.5)
}
xgb_search = RandomizedSearchCV(xgb, xgb_param, n_iter=20, cv=3, scoring="neg_mean_absolute_error", n_jobs=-1)
xgb_search.fit(X_train, y_train)
xgb_pred = xgb_search.best_estimator_.predict(X_test)
```

Model dasar XGBoost dibangun menggunakan XGBRegressor dengan `tree_method="hist"` yang mempercepat proses komputasi melalui teknik histogram-based splitting. Selanjutnya, ruang pencarian hyperparameter didefinisikan melalui beberapa parameter penting, yaitu jumlah pohon (`n_estimators`), tingkat pembelajaran (`learning_rate`), kedalaman maksimum pohon (`max_depth`), serta proporsi sampel dan fitur yang digunakan pada setiap pohon (`subsample` dan `colsample_bytree`). `RandomizedSearchCV` kemudian dijalankan dengan 20 percobaan acak dan 3-fold cross-validation, serta menggunakan *negative mean absolute error* sebagai metrik evaluasi. Setelah proses pencarian selesai, model terbaik yang ditemukan diterapkan untuk memprediksi data uji menggunakan `best_estimator`. Berikut nilai setelah dilakukan Hyperparameter Tuning:

```
=== XGBoost ===
MAE : 24131.791015625
RMSE: 35565.56536876646
R2   : 0.8350905179977417
```

Hasil evaluasi model menunjukkan bahwa XGBoost memiliki performa yang lebih baik dibandingkan RandomForest. Model RandomForest menghasilkan nilai MAE sebesar 24.727, RMSE 38.834, dan R^2 0,803, yang menandakan bahwa model mampu menjelaskan sekitar 80% variasi pada variabel target. Namun, XGBoost memberikan peningkatan kinerja dengan MAE sebesar 23.685, RMSE 34.895, dan R^2 0,841, sehingga model ini mampu menjelaskan sekitar 84% variasi data. Perbedaan nilai MAE dan RMSE menunjukkan bahwa XGBoost lebih akurat dalam memprediksi harga, dengan kesalahan rata-rata yang lebih kecil dan penyimpangan prediksi yang lebih rendah dibandingkan RandomForest. Secara keseluruhan, XGBoost merupakan model terbaik pada penelitian ini karena memberikan tingkat error paling kecil dan kemampuan prediksi paling kuat.

- LGBM dan XGBoost

```

X_train_lgbm, X_test_lgbm, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
X_train_xgb, X_test_xgb, _, _ = train_test_split(
    X_xgb, y, test_size=0.2, random_state=42
)
lgbm = LGBMRegressor(
    n_estimators=3000,
    learning_rate=0.02,
    num_leaves=64,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)
xgb = XGBRegressor(
    n_estimators=3000,
    learning_rate=0.02,
    max_depth=8,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.05,
    reg_lambda=1,
    random_state=42
)

```

LightGBM mampu membaca data kategori secara langsung, sehingga fitur kategori cukup dideklarasikan sebagai tipe *category*. Namun XGBoost tidak menerima data bertipe kategori, sehingga perlu dibuat salinan dataset khusus dengan melakukan *encoding* menggunakan `.cat.codes`, yakni mengonversi setiap kategori menjadi nilai numerik. Setelah proses encoding, data kemudian dibagi menjadi data latih dan data uji dengan menggunakan `train_test_split`. Pembagian dilakukan dua kali karena LightGBM menggunakan dataset asli (berisi kategori), sedangkan XGBoost menggunakan dataset yang sudah diubah menjadi angka. Model LightGBM dibangun dengan parameter `n_estimators=3000`, *learning rate* kecil, dan pengaturan *subsample* untuk meningkatkan stabilitas prediksi. Sementara itu, model XGBoost juga menggunakan 3000 *trees*, dengan kedalaman maksimum (*max_depth*) sebesar 8 serta tambahan regularisasi `reg_alpha` dan `reg_lambda` untuk mengurangi risiko *overfitting*. Tahap ini memastikan bahwa kedua model mendapatkan format data yang sesuai dan siap digunakan untuk proses *training* serta integrasi ke dalam model *stacking*.

```

===== RMSE OPTIMIZED MODEL =====
MAE   : 24276.535895788118
RMSE  : 34134.77815697267
R2     : 0.7380437638895947

```

Hasil evaluasi menunjukkan bahwa model Stacking yang menggabungkan LightGBM dan XGBoost mampu memberikan performa prediksi yang cukup baik. Nilai MAE sebesar 24.276 mengindikasikan bahwa secara rata-rata kesalahan prediksi harga rumah berada di kisaran 24 ribu dolar dibandingkan nilai sebenarnya. Selanjutnya, nilai RMSE sebesar 34.134 menunjukkan bahwa kesalahan model yang memperhitungkan deviasi besar juga cukup terkontrol. Sementara itu, nilai R^2 sebesar 0.738 menandakan bahwa model mampu menjelaskan sekitar 73.8% variasi pada harga rumah, yang merupakan performa yang kuat untuk permasalahan regresi kompleks seperti prediksi harga properti. Secara keseluruhan, kombinasi dua model boosting melalui teknik stacking terbukti meningkatkan akurasi dan kestabilan prediksi.

2.6 Perbandingan

No	Algoritma	MAE	RMSE	R^2
1	Random forest (12)	24.515	37.781	0.813
	Random forest (81)	17.467	28.572	0.893
2	XGBoost (12)	24.136	36.465	0.826
	XGBoost (81)	15.685	25.721	0.913
3	Hypertuning RF+XGB (12)	RF : 24.727 XGB : 23.685	RF : 38.834 XGB : 34.895	RF : 0.803 XGB : 0.841
	Hypertuning RF+XGB (81)	RF : 18.857 XGB : 15.519	RF : 33.925 XGB : 24.443	RF : 0.849 XGB : 0.922
4	LGBM+XGBoost (12)	24.276	34.134	0.738
	LGBM+XGBoost (81)	15.105	25.267	0.916

Tabel menunjukkan performa beberapa algoritma regresi (Random Forest, XGBoost, model tuning, dan LGBM) pada dua kondisi fitur: 12 fitur dan 81 fitur. Evaluasi menggunakan tiga metrik utama: MAE, RMSE, dan R^2 .

a. Perbandingan Random Forest

Random Forest merupakan metode ensemble berbasis *bagging* yang menggabungkan banyak pohon keputusan untuk menghasilkan prediksi rata-rata. Pada penggunaan 12 fitur, performa model masih terbatas karena informasi yang dimanfaatkan relatif sedikit, sehingga akurasi prediksi belum optimal (MAE 24.515; RMSE 37.781; R^2 0.813). Ketika jumlah fitur ditingkatkan menjadi 81, model memperoleh lebih banyak variabel penting yang membantu meningkatkan kedalaman pohon dan variasi keputusan. Hal ini membuat performa meningkat signifikan (MAE 17.467; RMSE 28.572; R^2 0.893).

Peningkatan ini menunjukkan bahwa Random Forest sangat bergantung pada keberagaman fitur dan bekerja jauh lebih baik ketika diberikan data yang lebih kaya. Peningkatan jumlah fitur dari 12 ke 81 membuat performa Random Forest meningkat signifikan dengan detail sebagai berikut:

- MAE turun 28%
- RMSE turun 24%
- R^2 naik dari 0.813 $\rightarrow$ 0.893

b. Perbandingan XGBoost

XGBoost adalah model *gradient boosting* berbasis pohon yang secara bertahap membangun pohon baru untuk memperbaiki kesalahan dari pohon sebelumnya. Pada data 12 fitur, model sudah cukup baik (MAE 24.136; RMSE 36.465; R^2 0.826). Namun, ketika fitur meningkat menjadi 81, XGBoost mampu memanfaatkan interaksi antarvariabel dan menangkap pola non-linear yang lebih kompleks, sehingga terjadi peningkatan akurasi yang signifikan (MAE 15.685; RMSE 25.721; R^2 0.913). Hal ini membuktikan bahwa XGBoost lebih responsif terhadap penambahan fitur dibanding Random Forest karena ia mempelajari error secara iteratif, bukan sekadar voting pohon. XGBoost mengalami peningkatan performa terbesar dibandingkan algoritma lain saat fitur bertambah dengan detail sebagai berikut:

- MAE turun 35%
- RMSE turun 29%
- R^2 naik 0.826 $\rightarrow$ 0.913

c. Hyperparameter Tuning RF + XGB

Proses hypertuning bertujuan memaksimalkan performa model dengan menyesuaikan parameter-parameter kunci seperti *max_depth*, *min_samples_split*, *learning_rate*, dan jumlah pohon. Pada 12 fitur, hypertuning memberikan peningkatan kecil, namun belum sangat drastis karena keterbatasan informasi (RF MAE 24.727 $\rightarrow$ XGB MAE (23.685).

Pada dataset 81 fitur, hypertuning berdampak jauh lebih besar karena model dapat mengoptimalkan parameter berdasarkan variasi data yang lebih kompleks. XGBoost setelah hypertuning mencapai performa tinggi (MAE 15.519; RMSE 24.443; R^2 0.922), menunjukkan bahwa tuning memberikan manfaat maksimal ketika jumlah fitur memadai. Kombinasi Random Forest + XGBoost (sebagai perbandingan) memperlihatkan bahwa XGBoost masih unggul pada kedua skenario fitur.

d. Ensemble LGBM + XGBoost

Model ensemble final menggabungkan prediksi LightGBM dan XGBoost. LightGBM bekerja dengan *leaf-wise splitting*, yang membuatnya sangat efisien dan mampu menyesuaikan struktur pohon secara optimal pada data berfitur tinggi. Ketika

digabung dengan XGBoost—yang lebih stabil dan mampu mengoreksi error—model ensemble ini memadukan kelebihan keduanya.

Dengan 12 fitur, performanya masih moderat (MAE 24.276; RMSE 34.134; R^2 0.738). Namun pada dataset 81 fitur, ensemble ini menjadi model terbaik dari seluruh pengujian, dengan MAE 15.105, RMSE 25.267, dan R^2 0.916. Penggabungan dua model boosting membuat prediksi lebih halus, mengurangi overfitting salah satu model, dan meningkatkan generalisasi secara signifikan. Sedangkan pada 81 fitur, ensemble LGBM+XGB meningkat drastis dengan detail sebagai berikut:

- MAE merosot menjadi yang terendah di seluruh tabel
- RMSE sangat rendah
- R^2 mencapai 0.916, hampir menyamai XGB tuning (0.922)

Dari keseluruhan algoritma yang digunakan untuk membangun model, dapat disimpulkan bahwa XGBoost dengan Hyperparameter Tuning yang diuji pada keseluruhan fitur (81 fitur) memiliki model terbaik keseluruhan untuk prediksi harga rumah. Dengan nilai $R^2 = 0.922$ yang artinya memiliki kekuatan generalisasi dan kestabilan yang tinggi. Khususnya pada algoritma dengan Ensemble LGBM + XGB yang diuji dengan keseluruhan fitur (81 fitur). Dikatakan lemah sebab memiliki MAE = 15.105 dan RMSE = 25.267 meski nilai $R^2 = 0.916$ yang mendekati model terbaik.

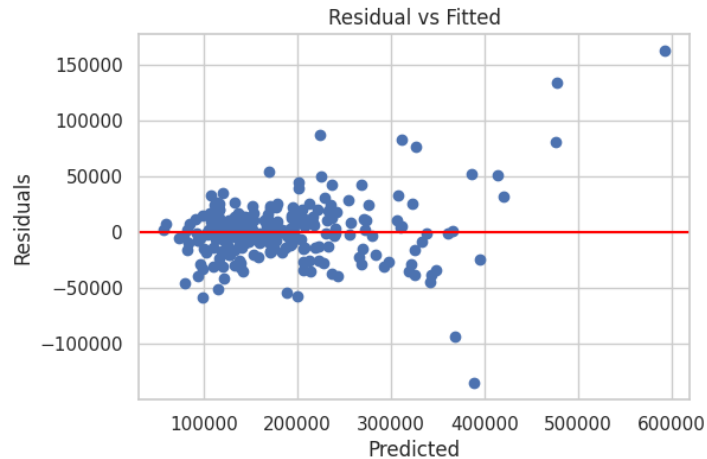
Jadi penambahan jumlah fitur dari 12 menjadi 81 memberikan peningkatan performa yang sangat signifikan pada seluruh model, terutama algoritma gradient boosting seperti XGBoost dan LightGBM. XGBoost menunjukkan peningkatan paling besar, dengan nilai R^2 naik dari 0.826 menjadi 0.913, dan mencapai puncak performa setelah hyperparameter tuning dengan $R^2 = 0.922$, menjadikannya model terbaik secara keseluruhan dalam hal kemampuan menjelaskan variansi data. Sementara itu, ensemble LGBM + XGBoost menjadi model dengan error terendah, memperoleh MAE = 15.105 dan RMSE = 25.267, menunjukkan bahwa kombinasi boosting model sangat efektif dalam menekan error absolut. Secara umum, hasil ini menegaskan bahwa model boosting modern yang diperkaya dengan fitur tinggi dan tuning parameter adalah pendekatan optimal dalam memprediksi harga rumah pada dataset ini.

2.7 Evaluasi Model

Evaluasi model dilakukan untuk memeriksa apakah model yang dibangun dengan algoritma tertentu sudah cukup baik untuk melakukan prediksi harga rumah yang bisa diterapkan dalam dunia industri. Evaluasi model dilakukan secara sederhana dengan menerapkan model pada dataset yang nantinya dilakukan visualisasi untuk menentukan langkah selanjutnya.

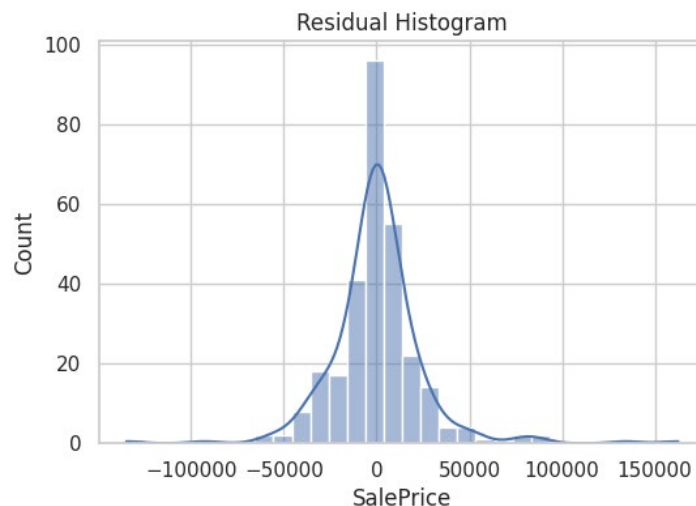
a. Seluruh Fitur

- Residual vs Fitted

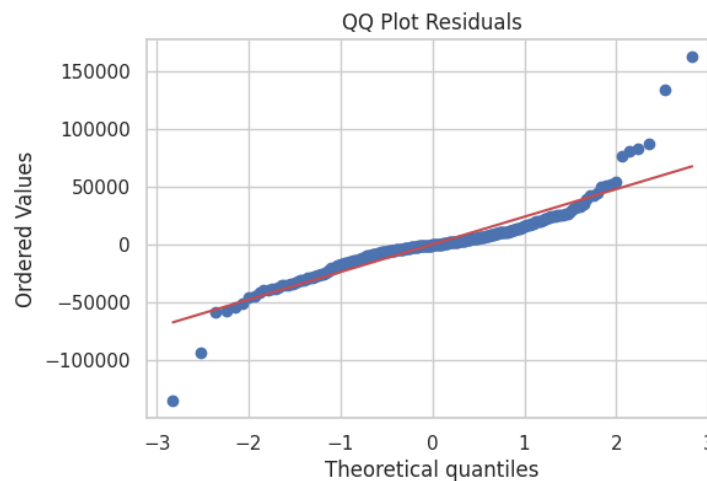


1. Titik-titik banyak mengelompok di sekitar garis merah (nol) yang menunjukkan bahwa sebagian besar prediksi model memiliki error kecil, sehingga model cukup akurat dalam memprediksi harga rumah pada sebagian besar data.
2. Sebaran residual yang cenderung melebar ke kanan (pola kipas/fan shape) yang menandakan adanya heteroskedastisitas, yaitu varians error yang meningkat ketika nilai prediksi semakin besar. Artinya, model semakin tidak stabil ketika memprediksi rumah dengan harga tinggi.
3. Adanya titik-titik yang jauh dari garis nol memiliki arti bahwa titik-titik ekstrem menunjukkan adanya overprediction (residual negatif besar) dan underprediction (residual positif besar) yang biasanya terjadi pada rumah dengan karakteristik yang sangat berbeda dari mayoritas data.
4. Titik ekstrem di bagian atas dan bawah grafik mengindikasikan bahwa model kesulitan menangkap pola pada data ekstrem, sehingga muncul error yang cukup besar.

- Residual Histogram

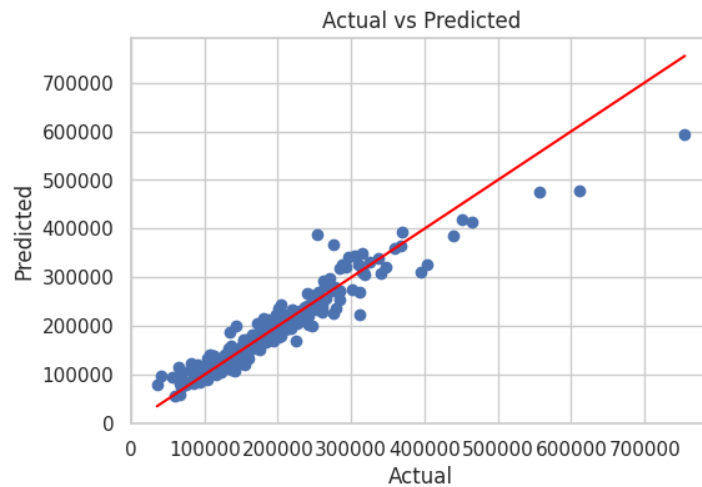


1. Bentuk histogram yang menyerupai lonceng (bell-shaped) memiliki arti bahwa distribusi residual mendekati distribusi normal, yang merupakan kondisi ideal dalam model regresi.
 2. Puncak histogram yang berada di sekitar nol menandakan bahwa sebagian besar error berpusat di sekitar nol, artinya prediksi model secara umum tidak bias (tidak terlalu overpredict atau underpredict).
 3. Ekor kiri dan kanan yang memanjang menunjukkan terdapat error ekstrem pada sebagian kecil data, yang biasanya berasal dari outlier atau nilai target yang sangat tinggi atau rendah. Pada kasus ini, outlier berasal dari target harga rumah yang tinggi memiliki nilai yang sangat sedikit.
 4. Distribusi sedikit tidak simetris sempurna yang mengindikasikan masih ada sedikit skewness, artinya model masih memiliki kecenderungan error berbeda antara underprediction dan overprediction.
- QQ Plot Residuals

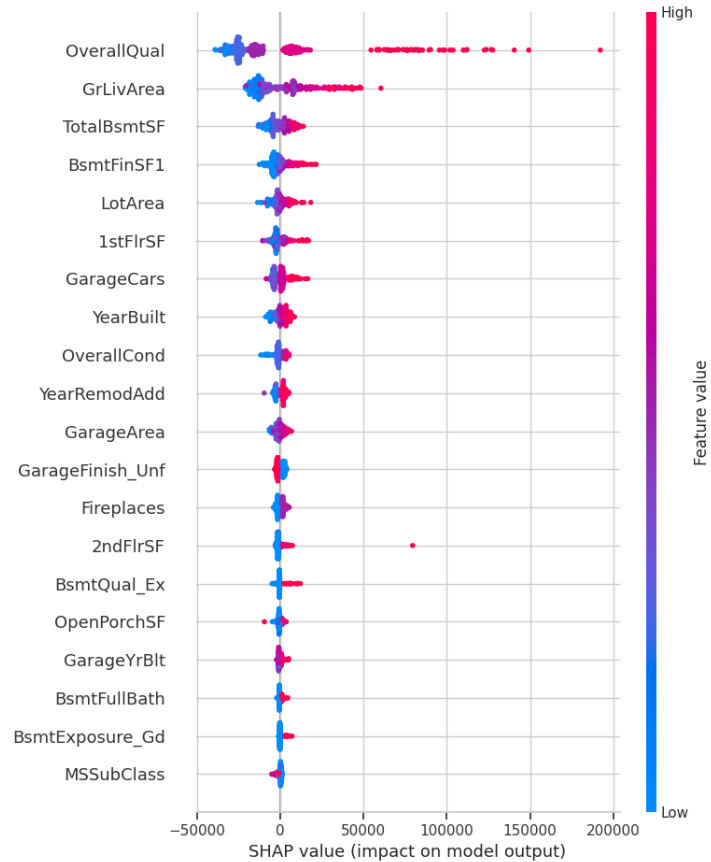


1. Titik-titik di bagian tengah mengikuti garis merah menunjukkan bahwa residual pada sebagian besar data mendekati distribusi normal. Artinya, asumsi normalitas error pada model regresi secara umum sudah terpenuhi.
2. Deviasi yang terlihat jelas di bagian ekor kiri dan kanan yang ditandai dengan titik-titik yang menjauh dari garis pada kedua ujung grafik artinya terdapat outlier dan error ekstrem. Dengan detail sisi kiri (ekor bawah), titik-titik berada jauh di bawah garis menunjukkan model terlalu tinggi dalam memprediksi beberapa data. Sedangkan disisi kanan (ekor atas), titik-titik jauh di atas garis menunjukkan model terlalu rendah dalam memprediksi beberapa data.
3. Pola melengkung di bagian ujung (tail curvature) menandakan bahwa distribusi residual tidak sepenuhnya normal dan memiliki ekor yang lebih berat (heavy tails), yang umum terjadi pada data harga properti.

- Actual vs Predicted



1. Sebagian besar titik mengelompok di sekitar garis merah diagonal yang menunjukkan bahwa prediksi model cukup akurat untuk mayoritas data.
 2. Pola linear yang jelas di bagian tengah memiliki arti model berhasil mempelajari hubungan utama antara fitur dan harga rumah.
 3. Penyebaran titik makin melebar pada nilai tinggi menandakan bahwa error meningkat pada harga rumah yang besar, sehingga model kurang presisi dalam memprediksi nilai ekstrem.
 4. Adanya titik-titik yang berada jauh dari garis merah menunjukkan adanya outlier atau properti dengan karakteristik unik yang sulit ditangkap oleh model.
- SHAP Value



1

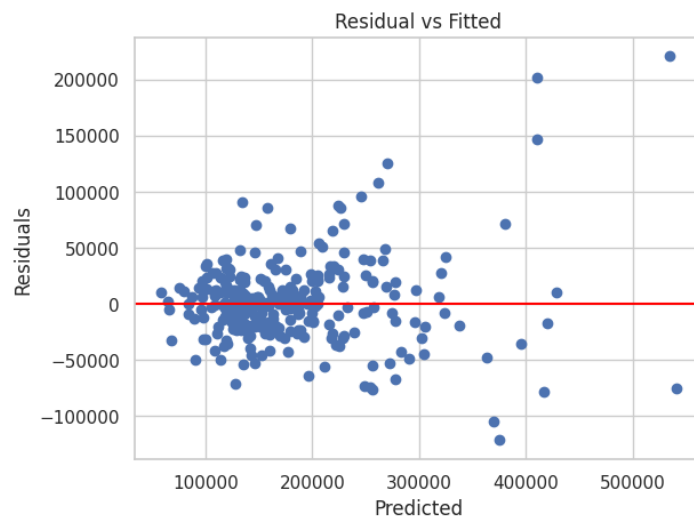
1. Fitur-fitur teratas memiliki pengaruh paling besar terhadap prediksi, seperti OverallQual, GrLivArea, dan TotalBsmtSF berada di urutan pertama. Ini menunjukkan bahwa kualitas bangunan, luas area hunian, dan luas basement adalah faktor yang paling dominan dalam memengaruhi harga rumah pada model.
2. Sebaran titik ke kanan dan kiri garis nol menunjukkan arah pengaruh prediksinya. Titik yang berada di sisi kanan (SHAP positif) berarti fitur tersebut menaikkan prediksi harga rumah, sedangkan titik di sisi kiri (SHAP negatif) berarti fitur tersebut menurunkan prediksi harga.
3. Warna titik menunjukkan besar kecilnya nilai fitur, dengan detail titik berwarna merah menunjukkan nilai fitur yang tinggi, sedangkan biru menunjukkan nilai fitur yang rendah. Ini memperlihatkan hubungan antara nilai fitur dan dampaknya terhadap harga.
4. OverallQual menunjukkan pola yang sangat kuat. Nilai tinggi (titik merah) banyak berada jauh di sisi kanan, menandakan bahwa kualitas bangunan yang tinggi secara konsisten meningkatkan harga rumah secara signifikan. Sedangkan nilai rendah (titik biru) dominan di sisi kiri, yang berarti kualitas rendah secara konsisten menurunkan nilai prediksi.
5. GrLivArea memperlihatkan hubungan yang hampir linear. Semakin luas area hunian, titik-titik merah cenderung bergerak ke sisi kanan, menandakan

peningkatan harga. Sebaliknya, rumah dengan area kecil cenderung berada di sisi kiri.

6. Fitur-fitur struktural seperti TotalBsmtSF, 1stFlrSF, dan GarageCars menunjukkan pola kontribusi positif yang jelas. Semakin besar nilai fitur, semakin besar peningkatan harga yang diprediksi.
7. Sebagian fitur memiliki pengaruh kecil dan terpusat di sekitar nol, seperti MSSubClass, GarageFinish_Unf, dan beberapa fitur basement lain memiliki titik-titik yang berkumpul dekat nol, menandakan bahwa pengaruhnya terhadap prediksi relatif kecil.
8. Adanya titik-titik ekstrem di beberapa fitur yang jauh dari nol menunjukkan adanya observasi tertentu di mana fitur tersebut memberikan dampak yang sangat besar pada prediksi, yang biasanya berkaitan dengan properti yang sangat unik atau bernilai sangat tinggi.

b. 12 Fitur Berdasarkan Lokasi

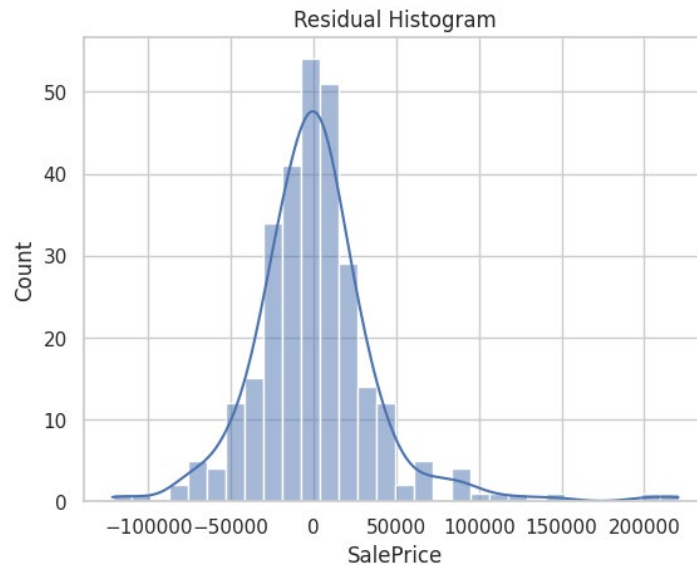
- Residual vs Fitted



1. Setiap titik biru mewakili satu rumah dari data yang diuji. Sumbu X berisi nilai prediksi harga rumah dari model. Sedangkan Sumbu Y berisi residual (error) yang didapat dari rumus $y_{\text{actual}} - y_{\text{pred}}$.
2. Sebagian besar titik berada di sekitar garis merah (residual = 0) yang menunjukkan bahwa model tidak memiliki bias besar prediksi rata-rata tidak terlalu tinggi atau terlalu rendah.
3. Titik-titik menyebar lebih lebar pada nilai prediksi yang tinggi yang beraryi prediksi untuk rumah mahal akan memiliki error yang lebih besar dan lebih sulit diprediksi dikarenakan terjadi outlier. Secara statistik hal ini disebut heteroskedastisitas, yaitu varians error meningkat saat harga rumah meningkat.

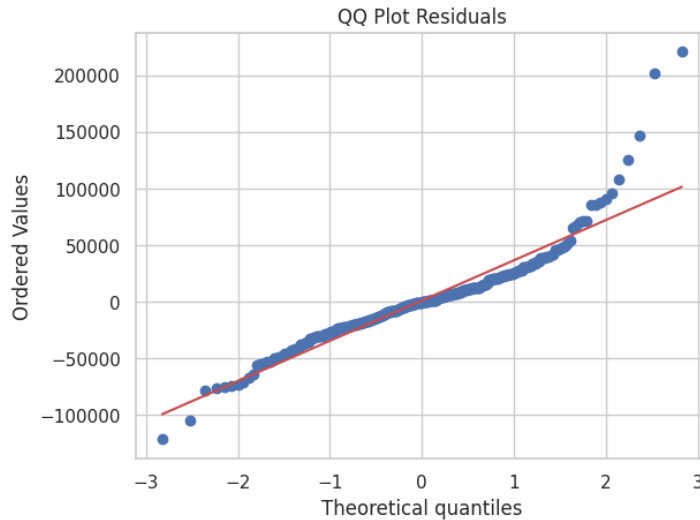
4. Beberapa outlier tampak jauh di atas atau di bawah garis yang menunjukkan data rumah-rumah dengan karakteristik sangat tidak umum (misalnya fitur sangat unik), sehingga model kurang mampu memprediksi secara presisi.
5. Pola tidak membentuk kurva atau bentuk sistematis yang berarti model sudah bagus. Namun, jika terbentuk pola seperti kurva, berarti model salah mempelajari hubungan.

- Residual Histogram



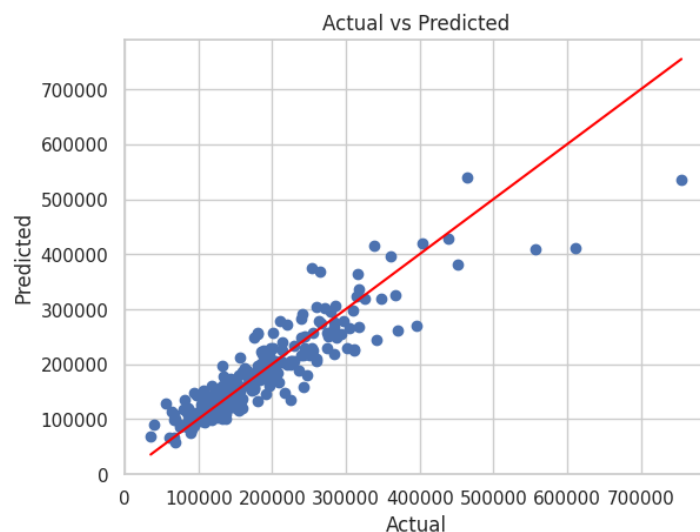
1. Histogram menunjukkan distribusi error model dimana setiap bar mewakili banyaknya rumah yang mempunyai error dalam range tertentu.
2. Bentuk histogram mendekati normal ditambah dengan smooth curve menunjukkan bahwa model cukup seimbang untuk memprediksi dengan error yang relatif kecil dan tidak condong pada satu arah sehingga tidak akan terjadi overpredict atau underpredict.
3. Puncak histogram berada dekat angka 0 residual bermakna bahwa prediksi dengan model mendekati dengan nilai sebenarnya dilihat dari MAE yang relatif kecil.
4. Histogram memiliki ekor panjang di kiri dan kanan yang berarti masih terdapat prediksi yang melesat meski persentasinya kecil, prediksi melesat ini akan umum terjadi pada rumah yang memiliki harga tinggi atau rumah dengan fitur yang sangat unik. Hal ini masih tergolong wajar sebab dataset memiliki banyak variasi fitur.
5. Distribusi tidak simetris 100% sempurna yang menunjukkan bahwa data aslinya memiliki kompleksitas yang tinggi serta model terlalu menangkap banyak pola.

- QQ Plot Residuals



1. Titik-titik di tengah mengikuti garis merah menandakan bahwa residual sebagian besar mendekati distribusi normal, yang merupakan asumsi yang baik untuk model regresi.
2. Deviasi besar di bagian ekor (atas dan bawah) dengan detail titik-titik di ekor bawah yang berarti model terlalu tinggi memprediksi harga dan titik-titik di ekor atas yang menandakan model terlalu rendah memprediksi harga.
3. Outlier terlihat jelas melenceng dari garis yang menunjukkan bahwa data sebenarnya memiliki nilai ekstrim (rumah premium atau rumah sangat murah), sehingga model tidak bisa sepenuhnya menangkap variasi ekstrem ini.

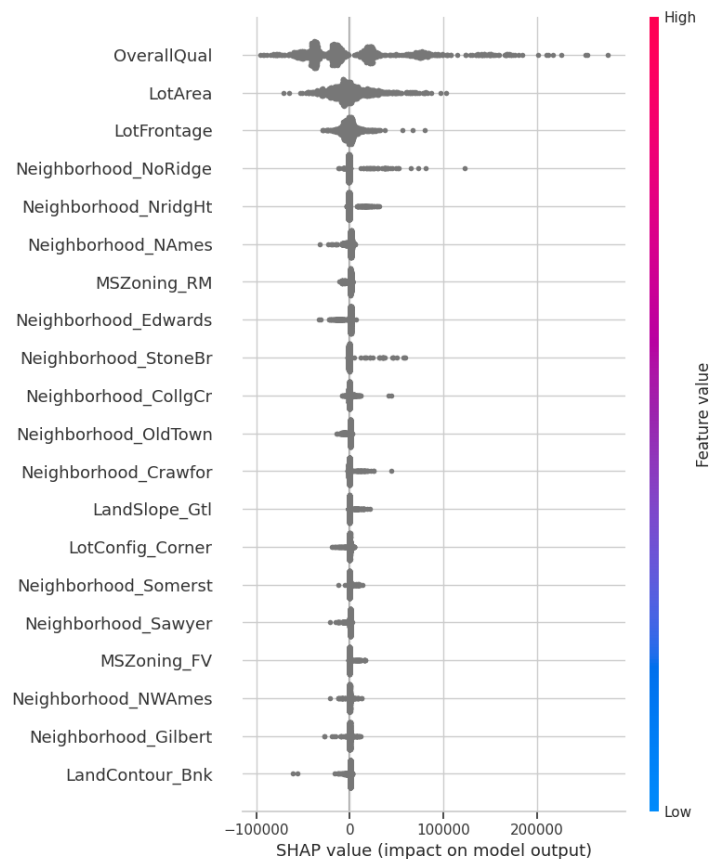
- Actual vs Predicted



1. Titik-titik di tengah mengikuti garis merah menunjukkan bahwa sebagian besar prediksi model sangat dekat dengan nilai aktual. Artinya, model mampu menangkap pola umum hubungan antara fitur dan harga rumah dengan baik, terutama pada rentang harga menengah.

- Sebaran yang rapat di sekitar garis diagonal mengindikasikan bahwa error (residual) relatif kecil dan stabil pada sebagian besar data, sehingga performa model dapat dikatakan cukup konsisten dalam kondisi normal.
- Deviasi besar di bagian ekor (atas dan bawah), dengan detail titik-titik yang berada jauh dari garis merah di bagian atas menunjukkan adanya error positif ekstrem, di mana model terlalu rendah dalam memprediksi nilai aktual (underestimation). Sedangkan titik-titik yang jauh di bagian bawah garis merah menunjukkan error negatif ekstrem, di mana model terlalu tinggi dalam memprediksi nilai aktual (overestimation).
- Outlier terlihat jelas melenceng dari garis menandakan adanya data dengan karakteristik ekstrem, seperti rumah dengan harga sangat tinggi (premium) atau sangat rendah. Model kesulitan menangkap pola dari data ekstrem ini karena jumlahnya sedikit atau memiliki karakteristik yang sangat berbeda dari mayoritas data.
- Pola penyebaran yang semakin melebar pada nilai tinggi mengindikasikan bahwa ketidakpastian prediksi meningkat seiring bertambahnya nilai aktual, yang dapat mencerminkan adanya heteroskedastisitas, yaitu varians error yang tidak konstan.

- SHAP Value



1. Fitur dengan pengaruh paling besar berada di bagian atas grafik menunjukkan tingkat kepentingan terhadap prediksi model. *OverallQual* berada di posisi teratas, yang berarti kualitas keseluruhan bangunan adalah faktor paling dominan dalam menentukan harga rumah.
2. Sebaran titik ke kanan dan kiri dari garis nol (0), dengan detail titik di sisi kanan (nilai SHAP positif) yang menunjukkan fitur tersebut menaikkan prediksi harga, sedangkan titik di sisi kiri (nilai SHAP negatif) menunjukkan fitur tersebut menurunkan prediksi harga.
3. Titik berwarna merah (High) menunjukkan nilai fitur yang tinggi, sedangkan biru (Low) menunjukkan nilai fitur yang rendah. Hal ini memperlihatkan bagaimana nilai fitur berkorelasi dengan kenaikan atau penurunan harga.
4. *OverallQual* menunjukkan pola yang sangat jelas. Titik merah (nilai kualitas tinggi) banyak berada di sisi kanan, yang berarti rumah dengan kualitas bangunan tinggi secara konsisten meningkatkan harga prediksi. Sedangkan titik biru (kualitas rendah) cenderung berada di sisi kiri, yang berarti kualitas rendah menurunkan harga rumah.
5. *LotArea* dan *LotFrontage* memiliki pengaruh besar namun lebih menyebar menunjukkan bahwa luas lahan dan lebar muka tanah berpengaruh besar, tetapi dampaknya bervariasi tergantung kombinasi dengan fitur lain. Fitur *Neighborhood* (lokasi) menunjukkan efek spesifik area beberapa kategori seperti *Neighborhood_NoRidgeHt*, *Neighborhood_NridgHt*, *Neighborhood_Names* yang memiliki titik-titik yang dominan ke sisi kanan, menandakan bahwa rumah di lingkungan tersebut cenderung memiliki harga lebih tinggi. Sebaliknya, beberapa area lain menunjukkan titik ke kiri, yang artinya lokasi tersebut menurunkan prediksi harga.
6. Sebaran yang memanjang (long tail) pada beberapa fitur menunjukkan adanya pengaruh ekstrem dari beberapa fitur pada kasus tertentu, menandakan interaksi kompleks antarvariabel.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan evaluasi model pada proyek prediksi harga rumah menggunakan dataset *House Sale Data*, dapat disimpulkan bahwa penerapan algoritma *machine learning* mampu memberikan prediksi harga rumah yang akurat dan berbasis data. Proses eksplorasi data menunjukkan bahwa harga rumah dipengaruhi secara signifikan oleh fitur-fitur yang berkaitan dengan kualitas dan ukuran bangunan, khususnya OverallQual, GrLivArea, TotalBsmtSF, serta GarageCars. Hasil pengujian model menunjukkan bahwa penggunaan jumlah fitur yang lebih banyak (81 fitur) secara konsisten meningkatkan performa prediksi dibandingkan penggunaan 12 fitur berbasis lokasi saja.

Dari seluruh algoritma yang diuji, XGBoost dengan hyperparameter tuning memberikan performa terbaik dengan nilai R^2 sebesar 0,922, yang menunjukkan kemampuan model dalam menjelaskan variansi data harga rumah secara baik. Sementara itu, model LightGBM dan XGBoost mampu menghasilkan kesalahan prediksi absolut terendah, yang menandakan stabilitas prediksi yang tinggi. Evaluasi lanjutan melalui analisis residual dan SHAP Value membuktikan bahwa model telah mempelajari pola utama data secara tepat dan mampu memberikan interpretasi kontribusi fitur terhadap prediksi harga, meskipun masih terdapat keterbatasan dalam memprediksi

rumah dengan harga ekstrem. Dengan demikian, penelitian ini membuktikan bahwa algoritma *boosting* dengan jumlah fitur yang kaya dan konfigurasi parameter yang optimal merupakan pendekatan yang efektif untuk membangun sistem prediksi harga rumah yang akurat, transparan, dan dapat dijadikan sebagai standar pembandingan dalam penilaian properti berbasis data

3.2 Kendala

A. Kesulitan Menentukan Model yang Paling Cocok

Selama proses pengerjaan kami kesulitan dalam menentukan algoritma yang paling sesuai untuk memprediksi harga rumah. Beberapa model awal yang diuji, seperti Random Forest dan XGBoost tanpa tuning, menunjukkan perbedaan performa yang cukup signifikan sangat berbeda dan menghasilkan nilai yang lumayan tinggi. Selain itu, peningkatan performa tidak selalu konsisten, khususnya pada Random Forest yang justru mengalami penurunan akurasi setelah tuning. Kondisi ini menyebabkan proses pemilihan model terbaik membutuhkan banyak percobaan, evaluasi ulang, serta perbandingan mendalam antar algoritma sebelum diperoleh model yang benar-benar optimal.

B. Nilai RMSE dan MAE yang Relatif Tinggi dan Sulit Diturunkan

Pada saat menentukan model kami juga kesulitan dalam nilai evaluasi RMSE dan MAE yang tergolong cukup tinggi, terutama pada tahap awal pemodelan dan pada penggunaan 12 fitur berbasis lokasi. Pada awal pengerjaan, terdapat pemahaman bahwa nilai error yang baik seharusnya mendekati 0 atau berada pada kisaran 0, sekian. Namun, setelah dilakukan analisis lebih lanjut, diketahui bahwa nilai RMSE dan MAE sangat bergantung pada skala data target. Karena harga rumah berada pada skala puluhan hingga ratusan ribu dolar, maka nilai error yang berada pada kisaran belasan hingga puluhan ribu masih tergolong wajar dan dapat diterima.

C. Pengaruh Outlier dan Harga Ekstrem terhadap Akurasi Model

Pada dataset yang digunakan memiliki sejumlah rumah dengan harga sangat tinggi yang jumlahnya relatif sedikit dibandingkan rumah dengan harga menengah. Keberadaan outlier ini menyebabkan model mengalami kesulitan dalam mempelajari pola harga ekstrem, sehingga error prediksi pada segmen rumah mahal cenderung lebih besar. Hal ini berdampak langsung pada meningkatnya nilai RMSE yang sensitif terhadap kesalahan besar.

LAMPIRAN

Adetunji, A. B. (2022). House Price Prediction using Random Forest Machine Learning Technique. *Procedia Computer Science*, 199, 806-813.

<https://doi.org/10.1016/j.procs.2022.01.016>

Rinabi, H., & Ardiansyah, A. (2023). House price prediction model using Random Forest in Surabaya City. *TEM Journal*, 12(1), 126–132.

<https://doi.org/10.18421/TEM121-17>

Video presentasi progress

<https://youtu.be/hZjWvxMnEPs> (1) - Minggu ke 14

Video presentasi progress

<https://youtu.be/jU7Va5cHX0k> (2) - Minggu ke 15

Video final progress

<https://youtu.be/GpH00bEgEjE> (3) - Minggu ke 16